Code based cryptography

P. Véron

IMATH, TOULON, FRANCE

M2 parcours Mathématiques-Informatique



Sign In SUBSCRIBE: Home Delivery | Digital Jobs | Real Estate | Rentals | Cars | Today's Paper | Discussions | Going Out Guide

Politics | Opinions | Local | Sports | National | World | Business | Tech | Lifestyle | Entertainment | Jobs | Real Estate | More

The Washington Post

77° Washington, DC Edition: U.S. | Regional Make us your homepage

In the News Royal baby Phil Mickelson NSA Helen Thomas Trayvon Martin Bay Bridge

Digital subscriptions starting at $0.99. Click here to subscribe.

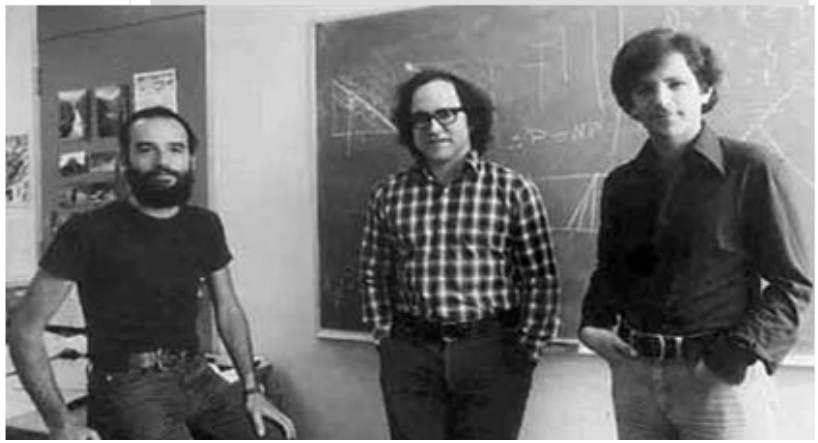
RSA is no more secure !!

Zachary A. Goldfarb

Computer scientists claim that they can totally break the RSA security technology used to protect everything from media players to smartphones and ecommerce servers.

- RSA story
- The decline of RSA






Sign In
SUBSCRIBE: Home Delivery | Digital
Jobs | Real Estate | Rentals
Politics | Opinions | Local | Sports | National | World | Business | Tech | Lifestyle
77° Washington, DC
In the News
Royal baby
Phil Mickelson
NSA
RSA is no more secure !!
SUBSCRIBE AND GET 1 MONTH FREE
TRY A FLEXIBLE MONTHLY SUBSCRIPTION
THE WALL STREET JOURNAL.
U.S. EDITION
Home
World
U.S.
New York
Business
Tech
Markets
Market Data
Opinion
Life & Culture
Real Estate
Management
Video
WSJ Startup of the Year
Alerts & Newsletters
Columns
Blogs
Heard on the Street
Small Biz
Wealth Management
QUICKLINKS: Today's Paper
Computer scientists provoke a huge market crash !!
The announcement of the total break of the RSA cryptosytem has exercised significant influences on the emergence of the actual global financial
Markets
Overview
U.S.
Europe
Asia
FX
Rates
Future
DJIA
Nasdaq
FTSE 100
Nikkei 225
Crude Oil
Gold
Market Data | MoneyBeat | Watchlist
$1 A WEEK for 12 WEEKS


Sign In
SUBSCRIBE: Home Delivery | Digital
Jobs | Real Estate | Rentals
Politics | Opinions | Local | Sports | National | World | Business | Tech | Lifestyle
SUBSCRIBE AND GET 1 MONTH FREE
TRY A FLEXIBLE MONTHLY SUBSCRIPTION
HOME PAGE | TODAY'S PAPER | VIDEO | MOST POPULAR | U.S. Edition
ROLEX
The New York Times
Tuesday, September 2, 2053
Last Update: 5:26 AM ET
Search
Try a Digital Subscription | Log In
Quantum computers are a reality
By STEVEN YACCINO and MICHAEL COOPER
It's a revolution. The last intel processor is the heart of a new generation of computers : quantum computers. It can manipulates qbits,the quantum analogue of the classical bits.In a classical system, a bit would have to be in one state or the other, but
huge crash !!
The announcement ... break of the RSA crypto... has exercised significant influences on the emergence of the actual global financial ...
56 min ago
intel Core Quantum
QUBIT ENTANGLED
Follow Us
Personalize Your W
The Opinion Pages
To Make a Safe City Safer
By THE EDITORIAL BOARD
Candidates for New York City mayor pledge more officers on patrol, better tactics and stronger community bonds.
MORE IN OPINION
Editorial: The Good Guy Wins
The Great Divide: Who's Your Daddy?
OP-ED COLUMNISTS
Krugman: Detroit, the New Greece
Keller: New York Is Not Detroit. But ...
OP-ED CONTRIBUTOR
Our Coming Food Crisis
By GARY PAUL NABHAN
Farmers must adapt to the new reality of scorching summ...


New York Tim
Tuesday, September 2, 2053
Last Update: 5:26 AM ET
To M
Safer

Post-Quantum Cryptography

Most Popular public-key cryptosystems can be broken in polynomial time by quantum computers using Shor's algorithm (1995)

- RSA,

Post-Quantum Cryptography

Most Popular public-key cryptosystems can be broken in polynomial time by quantum computers using Shor's algorithm (1995)

- RSA,
- ECC,

Post-Quantum Cryptography

Most Popular public-key cryptosystems can be broken in polynomial time by quantum computers using Shor's algorithm (1995)

- RSA,
- ECC,
- DSA,

Post-Quantum Cryptography

Most Popular public-key cryptosystems can be broken in polynomial time by quantum computers using Shor's algorithm (1995)

- RSA,
- ECC,
- DSA,
- ECDSA,

Post-Quantum Cryptography

Most Popular public-key cryptosystems can be broken in polynomial time by quantum computers using Shor's algorithm (1995)

- RSA,
- ECC,
- DSA,
- ECDSA,
- . . .

Post-Quantum Cryptography

Most Popular public-key cryptosystems can be broken in polynomial time by quantum computers using Shor's algorithm (1995)

- RSA,
- ECC,
- DSA,
- ECDSA,
- . . .

Cost of decryption attacks for the discrete logarithm problem

Size of the key	1024	2048	4096
Classic computer	10^5 years	5.10^{15} years	3.10^{29} years
Quantum computer	4,5 minutes	36 minutes	4,8 hours

Post-Quantum Cryptography

We need cryptosystems :

- running on conventionnal computers,

Post-Quantum Cryptography

We need cryptosystems :

- running on conventionnal computers,
- secure against quantum computers,

Post-Quantum Cryptography

We need cryptosystems :

- running on conventionnal computers,
- secure against quantum computers,

Post-Quantum Cryptography

We need cryptosystems :

- running on conventionnal computers,
- secure against quantum computers,

Lattice-based cryptography,

Post-Quantum Cryptography

We need cryptosystems :

- running on conventionnal computers,
- secure against quantum computers,

Lattice-based cryptography, Multivariate quadratic equations cryptography,

Post-Quantum Cryptography

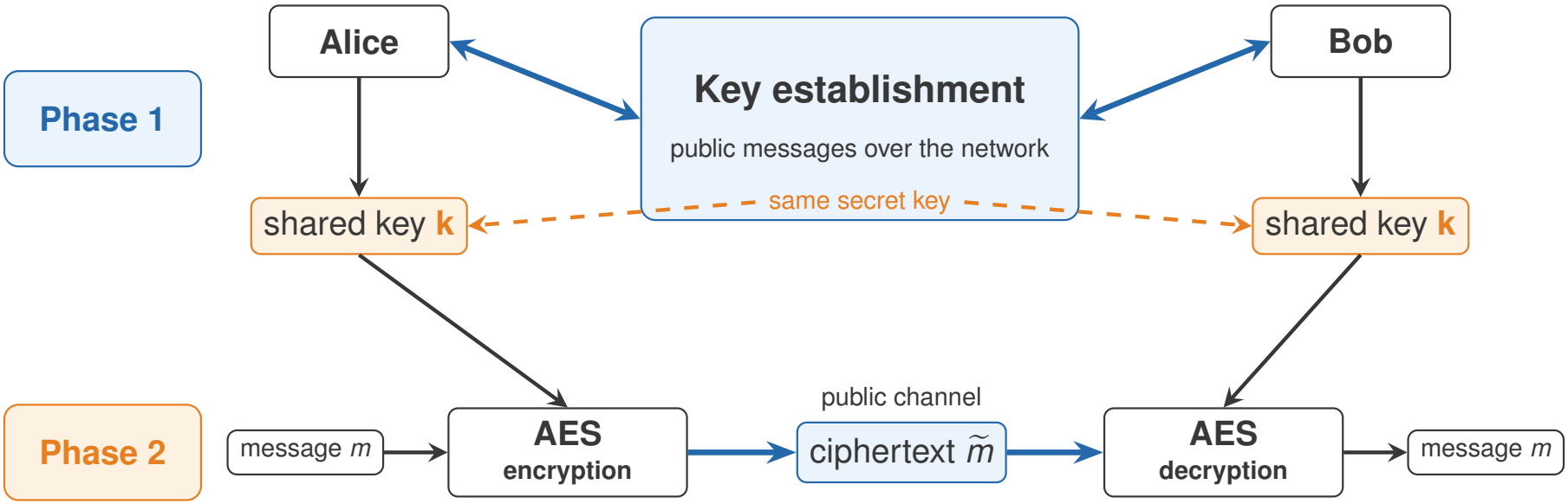
We need cryptosystems :

- running on conventionnal computers,
- secure against quantum computers,

Lattice-based cryptography, Multivariate quadratic equations cryptography,
Code based cryptography

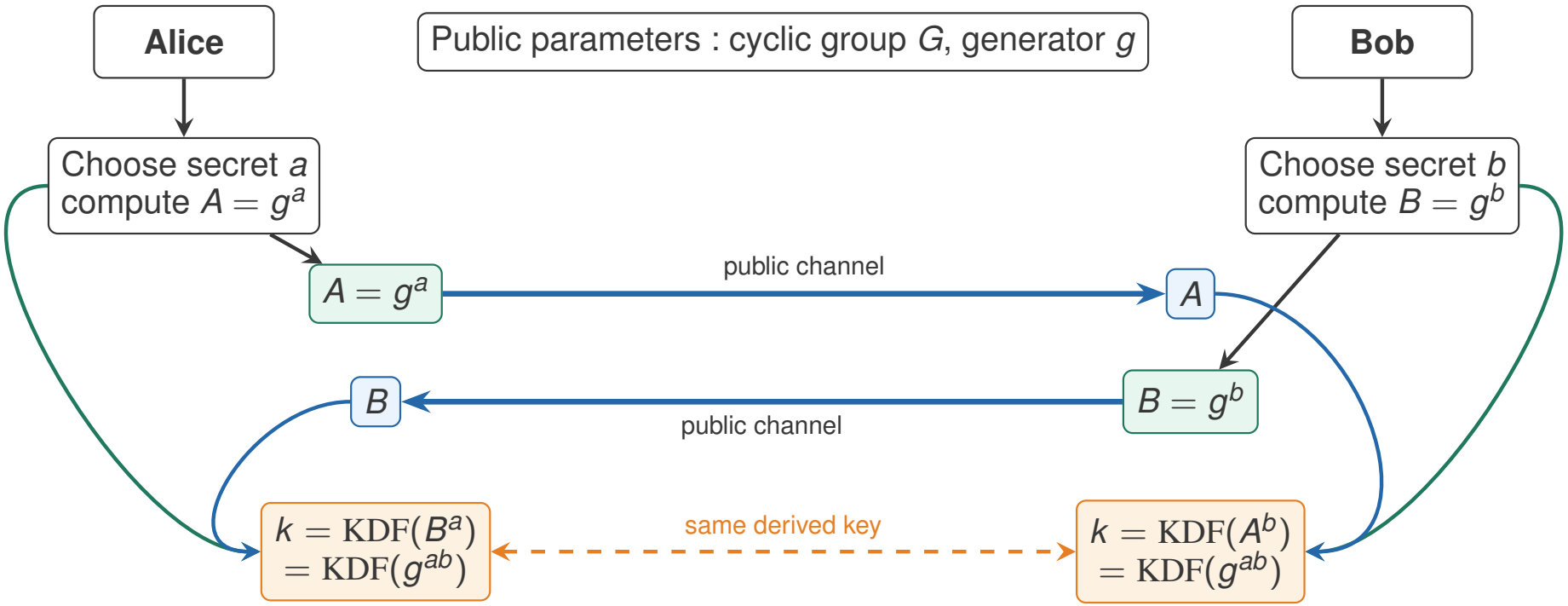
Secure Communication : overview

Secure communication has two phases



Takeaway: This presentation focuses on Phase 1 : modern ways for Alice and Bob to establish the same secret key.

Reminder : Diffie–Hellman key exchange



The basic exchange does not authenticate Alice or Bob.

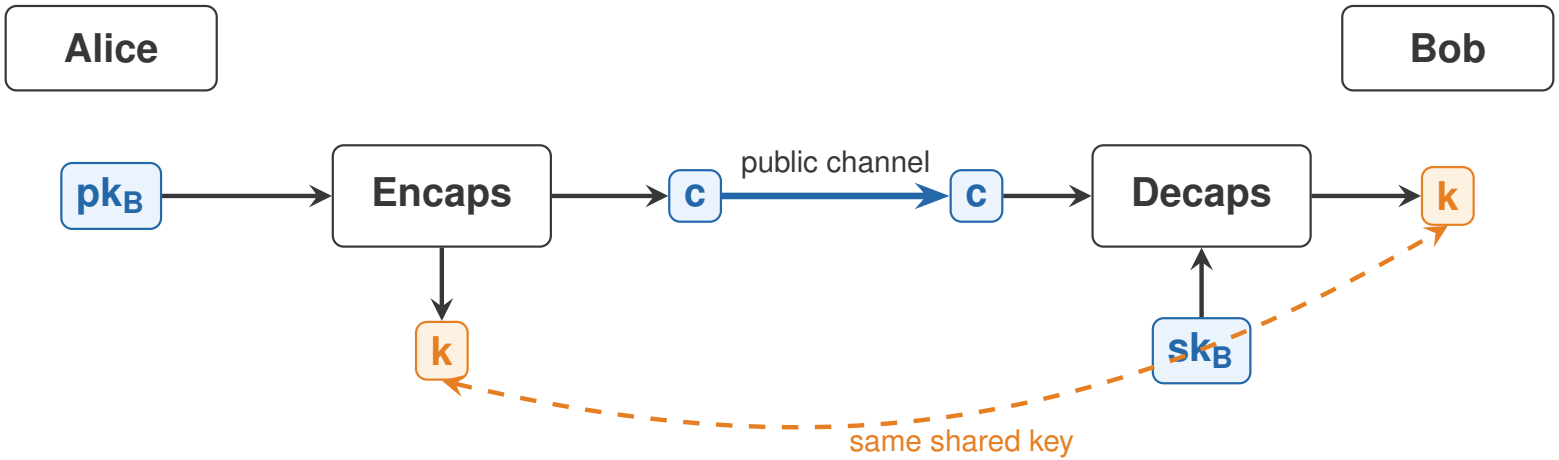
Key Encapsulation Mechanism

What is a KEM ?

- **KEM = Key Encapsulation Mechanism.**
- A KEM is a triple of algorithms :

$$\mathrm{KeyGen} \to (pk, sk), \qquad \mathrm{Encaps}(pk) \to (k, c), \qquad \mathrm{Decaps}(sk, c) \to k.$$

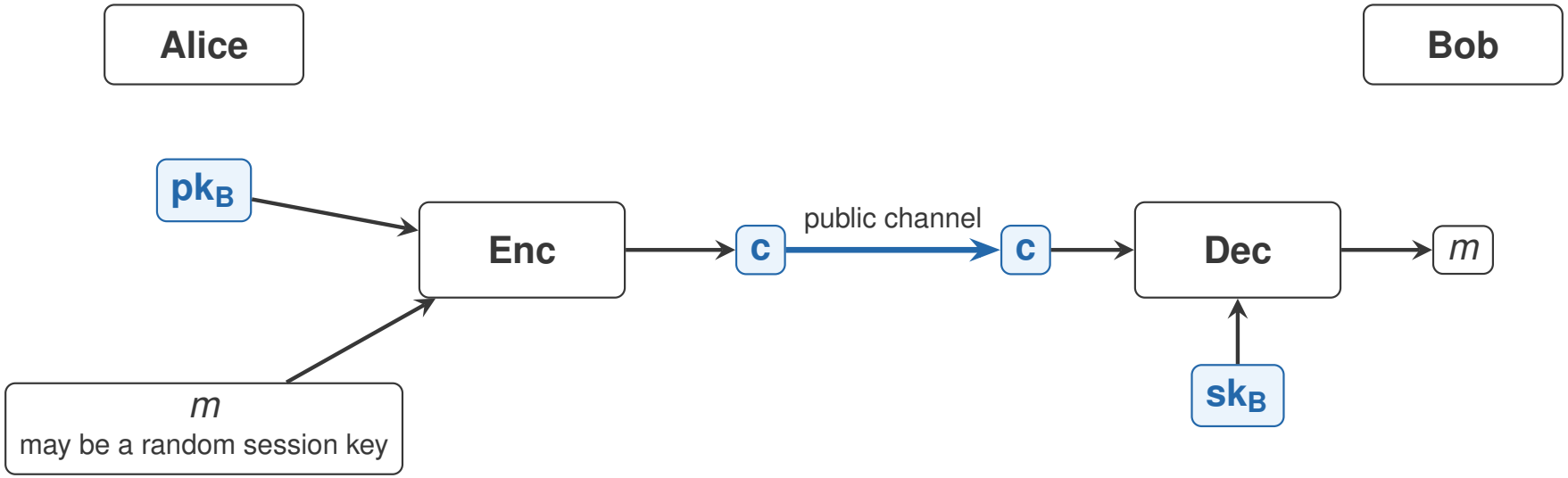
  Here k is a candidate shared key and c is a public capsule.
- Alice does **not** choose the key : Encaps generates it. Alice chooses the recipient ; both parties derive the same k.



Takeaway: A KEM is a mechanism whose **only purpose** is to deliver the same random key to two parties.

Reminder : how PKE works

- **PKE = Public-key encryption .**
- Public-key encryption protects whatever plaintext the sender wants to send.



KEM vs PKE : where does the key come from ?

PKE

- Alice picks a random key.
- She encrypts that chosen key under pk_B.

KEM

- Encaps generates the key.
- Alice only picks the recipient.

- Any PKE can be turned into a KEM by the standard transformation :

$$\begin{aligned} \text{Encaps}(pk): &\quad k \xleftarrow{\$} \mathcal{K}, \quad c \leftarrow \text{Enc}(pk, k), \quad \text{return } (k, c), \\ \text{Decaps}(sk, c): &\quad \text{return } k \leftarrow \text{Dec}(sk, c). \end{aligned}$$

Takeaway: The transformation works, but a KEM designed from scratch is a simpler object to define, build, and prove. A KEM **does not necessarily need** an encryption or decryption function.

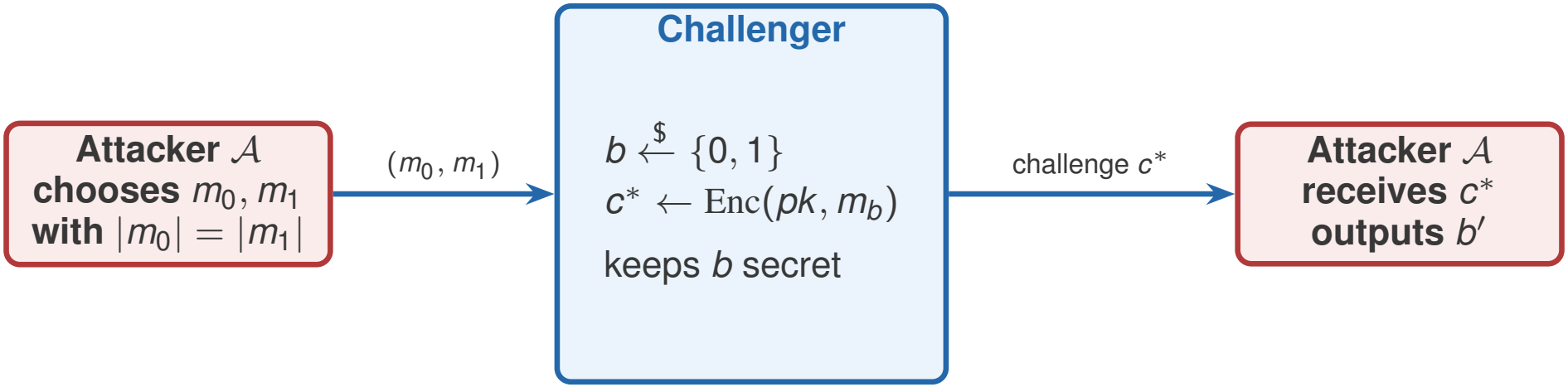
Using RSA as a KEM

- Building a KEM from RSA means : encrypt a freshly drawn key (e.g. a 128-bit AES key) under the RSA public key.
- The key must be padded (and properly) : use randomized, verifiable padding such as OAEP.
- Naive padding (filling with zeros for example) breaks security.

Takeaway: Modern KEMs are designed to output a key of exactly the required length without using any secure padding.

Security proofs : KEMs are easier to prove

PKE security : the IND-CPA game



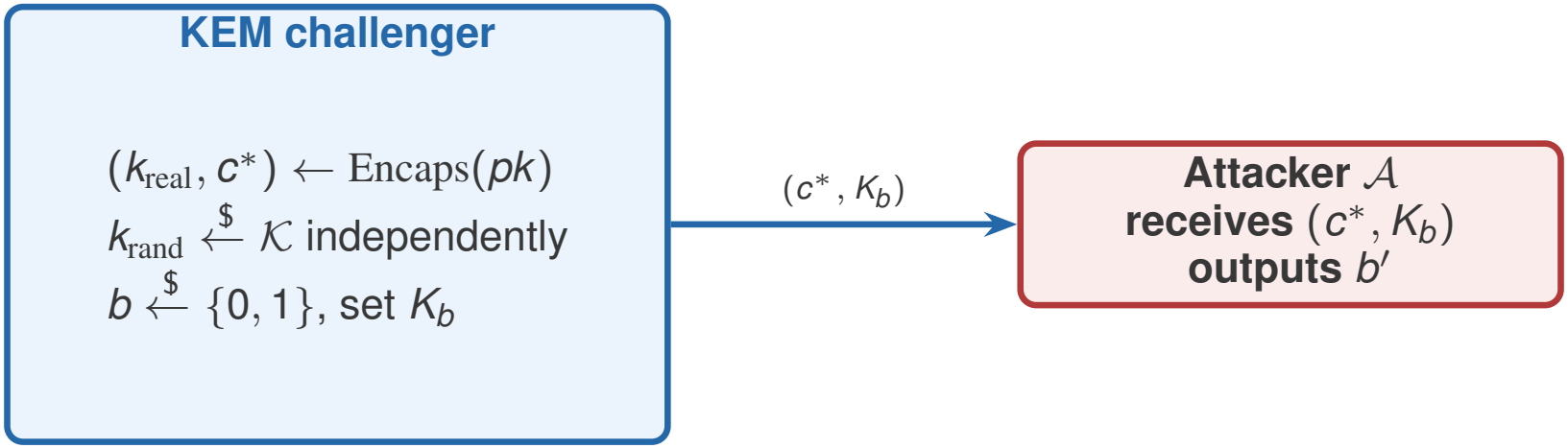
- **IND-CPA** means *indistinguishability under chosen-plaintext attack*.
- Security requires

$$\mathrm{Adv}_{\mathcal{A}}^{\text{IND-CPA}} = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

to be negligible.

Takeaway: The ciphertext must not reveal any information on the plaintext.

KEM security : real key or independent random key ?



Real world ($b = 0$)

$$K_0 = k_{\text{real}}$$

The key and capsule come from the same encapsulation.

Random world ($b = 1$)

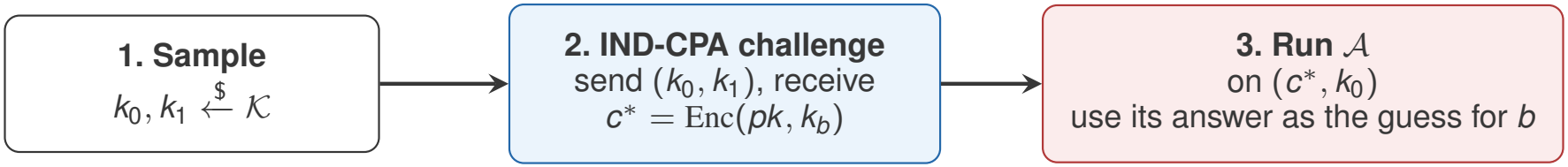
$$K_1 = k_{\text{rand}}$$

The displayed key is independent of the capsule.

Takeaway: The claim is conditional secrecy : $(c, k_{\text{real}}) \approx (c, k_{\text{rand}})$, not merely that k_{real} is uniformly distributed. An attacker cannot compute if there is or not a link between c^* and K_b.

Why IND-CPA security implies KEM security

Assume a distinguisher $\mathcal{A}$ separates the real and random KEM worlds. Build an IND-CPA attacker $\mathcal{B}$:



If $b = 0$

$$(c^*, k_0) = \big(\mathrm{Enc}(pk, k_0), k_0\big)$$

This is the **real KEM world**.

If $b = 1$

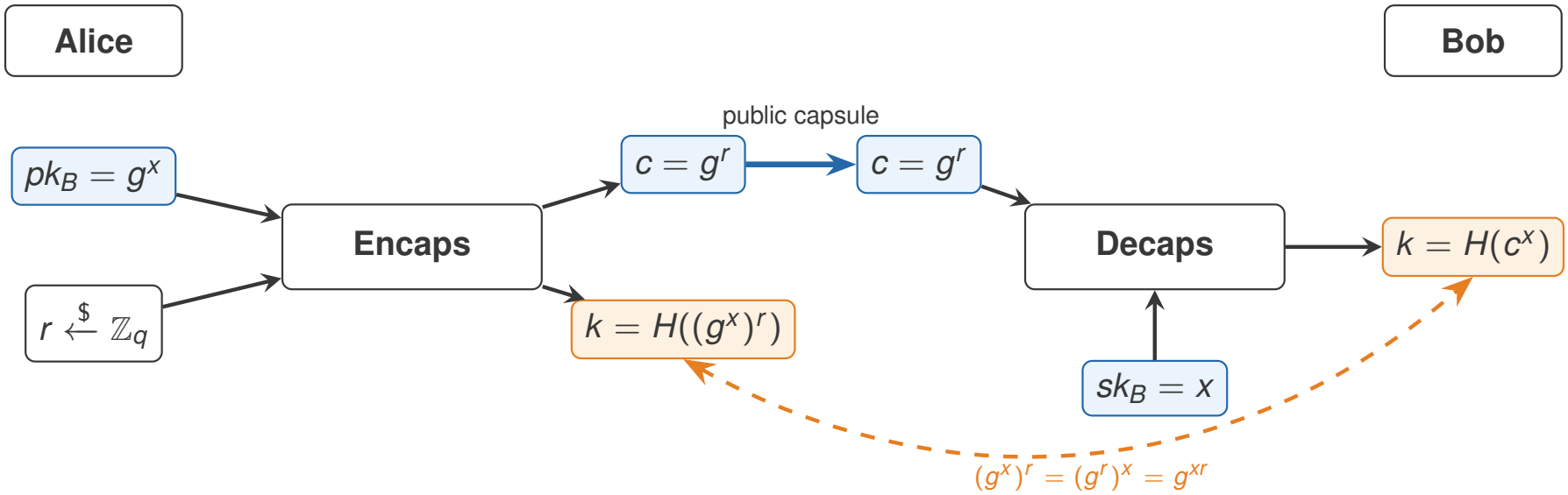
$$(c^*, k_0) = \big(\mathrm{Enc}(pk, k_1), k_0\big)$$

Here k_0 is independent of c^* : the **random KEM world**.

Takeaway: A KEM distinguisher would break IND-CPA. Therefore, IND-CPA of the PKE implies basic KEM indistinguishability.

DH-KEM : capsule and key from Diffie-Helmann protocol

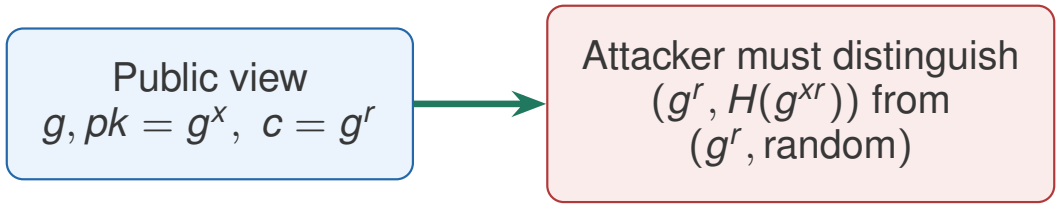
Bob samples $x \xleftarrow{\$} \mathbb{Z}_q$ and publishes $pk_B = g^x$. g is a public parameter.



DH-KEM security from CDH in the random-oracle model

CDH assumption

Given g, g^x and g^r, no efficient algorithm computes g^{xr} with non-negligible probability.



In the random-oracle model, $H(g^{xr})$ looks like an independent random string. Hence the only way to distinguish $H(g^{xr})$ from random is to compute g^{xr} first.

Hypothesis : if an attacker can break DH-KEM indistinguishability than he can solve CDH problem.

break DH-KEM indistinguishability $\Longrightarrow$ break CDH

Takeaway: Under CDH and the random-oracle model for H, the basic DH-KEM key is indistinguishable from random.

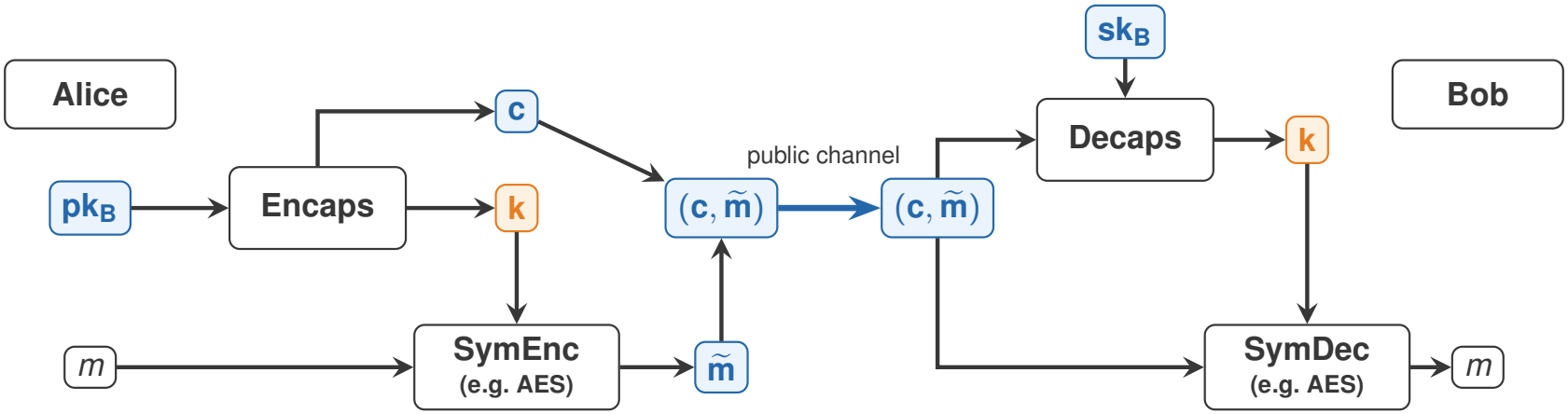
KEM vs Diffie–Hellman vs PKE

- DH and RSA are broken by Shor's algorithm on a sufficiently large quantum computer.
- A KEM is a **general abstraction** : an interface plus a security definition that can be satisfied by lattice-based primitives, code-based primitives, DH+KDF, RSA-OAEP, and other constructions.
- Generally, KEM security proofs are easier than PKE security proofs.
- Being a "KEM" does not itself make a scheme quantum-safe ; security depends on the underlying primitive.

Takeaway: DH is one instance ; KEM is the general envelope for key establishment — choose the envelope, then pick a quantum-resistant instance.

Using a KEM for encryption (hybrid / KEM-DEM)

- DEM : Data Encapsulation Mechanism
- The KEM establishes the key ; a symmetric cipher such as AES encrypts the actual data.
- Only the pair $(c, \tilde{m})$ travels over the channel.

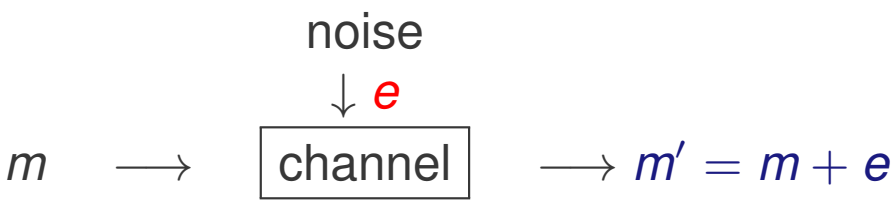


Takeaway: KEM for the key, symmetric encryption for the data — the standard pattern of modern (and post-quantum secure) encryption.

Coding Theory

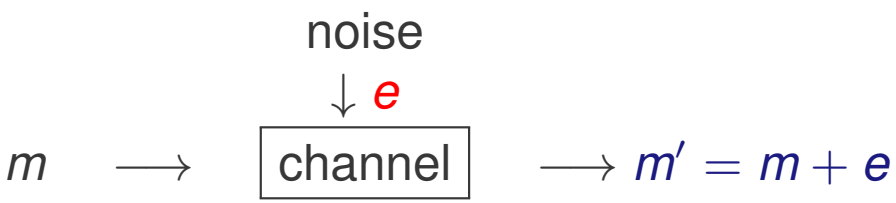
Coding theory : the essentials

Goal : protect again errors in a noisy channel



Coding theory : the essentials

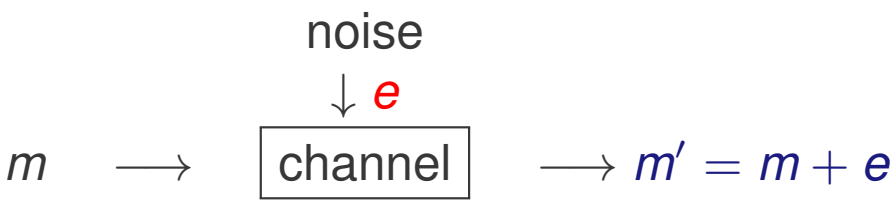
Goal : protect again errors in a noisy channel



How : add redundancy to the message

Coding theory : the essentials

Goal : protect again errors in a noisy channel

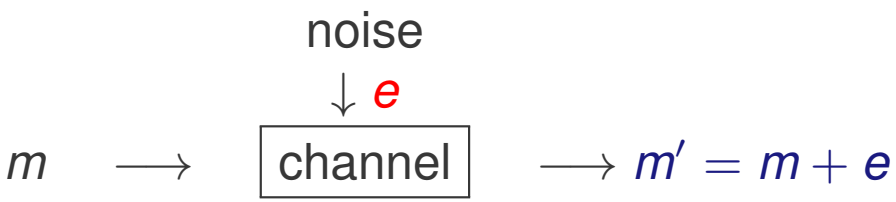


How : add redundancy to the message

Phonetic Alphabet			
A	Alpha	**N**	November
B	Bravo	**O**	Oscar
C	Charlie	**P**	Papa
D	Delta	**Q**	Quebec
E	Echo	**R**	Romeo
F	Foxtrot	**S**	Sierra
G	Golf	**T**	Tango
H	Hotel	**U**	Uniform
I	India	**V**	Victor
J	Juliet	**W**	Whiskey
K	Kilo	**X**	X-ray
L	Lima	**Y**	Yankee
M	Mike	**Z**	Zulu

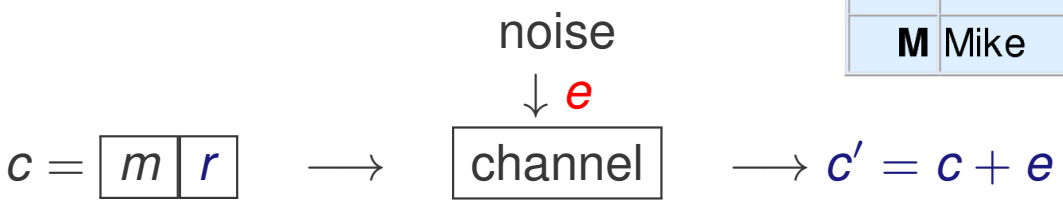
Coding theory : the essentials

Goal : protect again errors in a noisy channel



How : add redundancy to the message

Phonetic Alphabet			
A	Alpha	**N**	November
B	Bravo	**O**	Oscar
C	Charlie	**P**	Papa
D	Delta	**Q**	Quebec
E	Echo	**R**	Romeo
F	Foxtrot	**S**	Sierra
G	Golf	**T**	Tango
H	Hotel	**U**	Uniform
I	India	**V**	Victor
J	Juliet	**W**	Whiskey
K	Kilo	**X**	X-ray
L	Lima	**Y**	Yankee
M	Mike	**Z**	Zulu



Binary linear codes

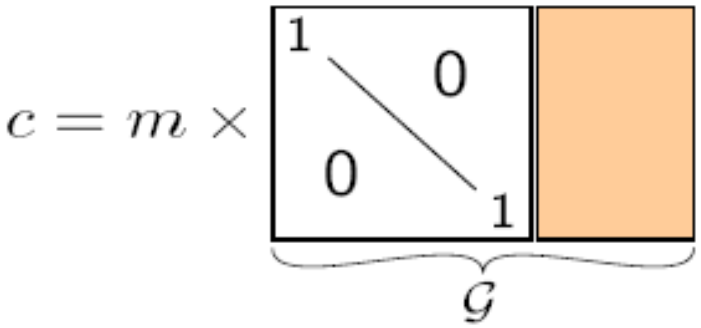
Binary $\longrightarrow$ binary channel

Binary linear codes

Binary $\longrightarrow$ binary channel
Linear $\longrightarrow$ redundancy linearly computed from the components of m

Binary linear codes

Binary $\longrightarrow$ binary channel
Linear $\longrightarrow$ redundancy linearly computed from the components of m

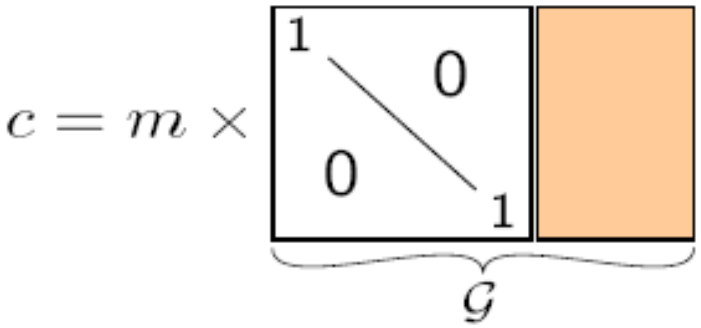


Binary linear codes

Binary $\longrightarrow$ binary channel
Linear $\longrightarrow$ redundancy linearly computed from the components of m

Definition

$G(k, n)$ is a *generator matrix* of the code $\mathcal{C}$.



Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
for codeword in C:
        print(codeword)
```

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
for codeword in C:
        print(codeword)
```

(**message**, *redudancy*)

(**0**, **0**, **0**, 0, 0)

(**1**, **0**, **0**, 1, 0)

(**0**, **1**, **0**, 1, 1)

(**1**, **1**, **0**, 0, 1)

(**0**, **0**, **1**, 1, 1)

(**1**, **0**, **1**, 0, 1)

(**0**, **1**, **1**, 0, 0)

(**1**, **1**, **1**, 1, 0)

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
print(C.length(),C.dimension())
```

```
5 3
```

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
msg = vector(GF(2),[1,1,0])
print(C.encode(msg))
```

(**1**, **1**, **0**, 0, 1)

A k-dimensional subspace

Definition

A binary linear code $\mathcal{C}$ of length n and dimension k is a k-dimensional subspace of $\mathbb{F}_2^n$.

A k-dimensional subspace

Definition

A binary linear code $\mathcal{C}$ of length n and dimension k is a k-dimensional subspace of $\mathbb{F}_2^n$.

The rows of G constitute a basis of this subspace.

Parity check matrix

A *parity check matrix* H of $\mathcal{C}$ is a $(n-k) \times n$ matrix which generates

$$\mathcal{C}^{\perp} = \{y \in \mathbb{F}_2^n \mid < x, y >= 0, \ \forall x \in \mathcal{C}\}$$

where

$$< x, y >= \sum_{i=1}^{n} x_i y_i$$

Parity check matrix

A *parity check matrix* H of $\mathcal{C}$ is a $(n-k) \times n$ matrix which generates

$$\mathcal{C}^{\perp} = \{y \in \mathbb{F}_2^n \mid < x, y >= 0,\ \forall x \in \mathcal{C}\}$$

where

$$< x, y >= \sum_{i=1}^{n} x_i y_i$$

i.e.,

Definition of a code $\mathcal{C}$ from H

$$\forall c = (c_1, \ldots, c_n) \in \mathbb{F}_2^n,\ c \in \mathcal{C} \Leftrightarrow H^t c = 0$$

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
print(C.generator_matrix())
print("")
print(C.parity_check_matrix())
```

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
print(C.generator_matrix())
print("")
print(C.parity_check_matrix())
```

$$\begin{pmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
H=C.parity_check_matrix()
for codeword in C:
    print(H*codeword)
```

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
H=C.parity_check_matrix()
for codeword in C:
    print(H*codeword)
```

(0, 0)

(0, 0)

(0, 0)

(0, 0)

(0, 0)

(0, 0)

(0, 0)

(0, 0)

From generator matrix to parity check matrix

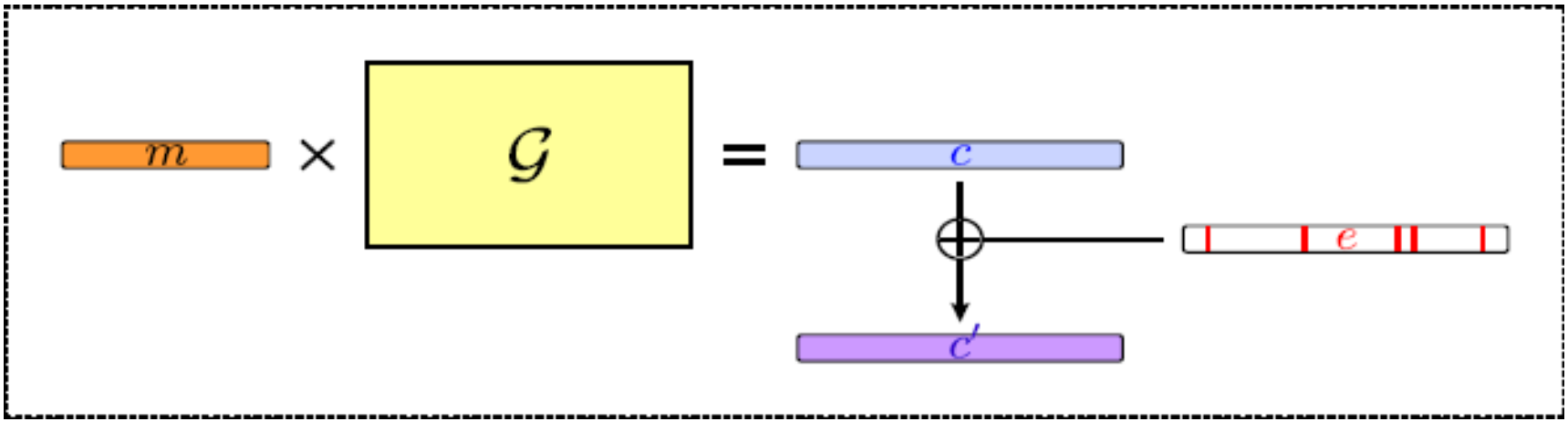
If $G = (I_k|A)$, where A is a $(k, n-k)$ binary matrix, is a generator matrix of a (n, k) code $\mathcal{C}$, then

$$H = ({}^tA|I_{n-k})$$

is a parity check matrix of $\mathcal{C}$.

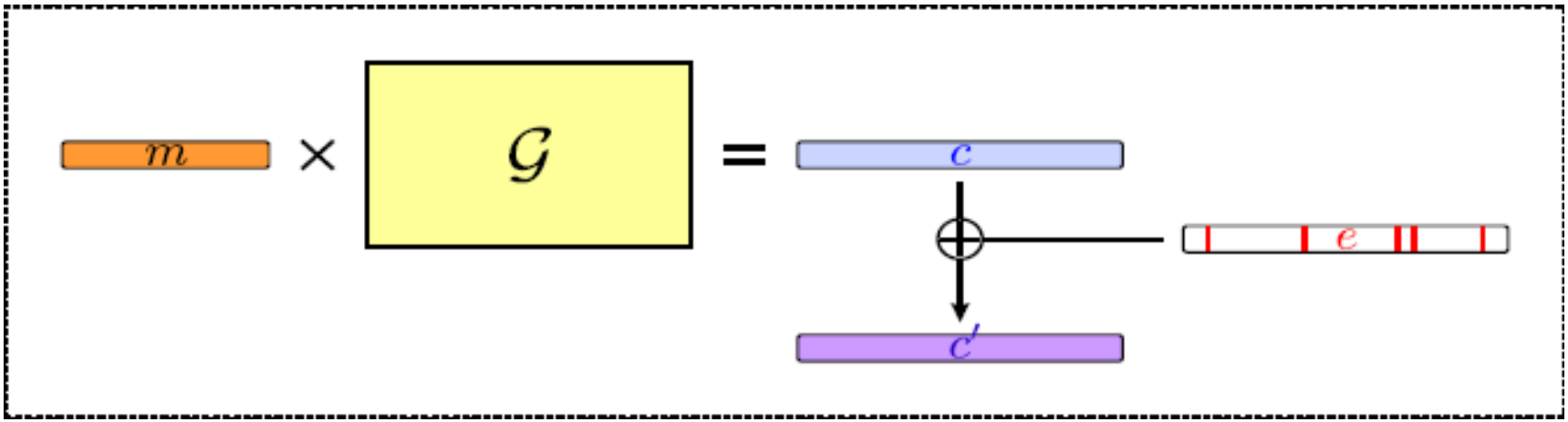
The syndrome decoding problem

How to compute c from c' ?



The syndrome decoding problem

How to compute c from c' ?



Definition

The Hamming weight of e, $\omega(e)$, is the number of non-zero coordinates of e.

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
V = VectorSpace(GF(2), 3)
for mess in V:
    codeword = C.encode(mess)
print(mess,codeword,codeword.hamming_weight())
```

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
V = VectorSpace(GF(2), 3)
for mess in V:
    codeword = C.encode(mess)
print(mess,codeword,codeword.hamming_weight())
```

(0, 0, 0)	(**0**, **0**, **0**, **0**, **0**)	0
(1, 0, 0)	(**1**, **0**, **0**, **1**, **0**)	2
(0, 1, 0)	(**0**, **1**, **0**, **1**, **1**)	3
(1, 1, 0)	(**1**, **1**, **0**, **0**, **1**)	3
(0, 0, 1)	(**0**, **0**, **1**, **1**, **1**)	3
(1, 0, 1)	(**1**, **0**, **1**, **0**, **1**)	3
(0, 1, 1)	(**0**, **1**, **1**, **0**, **0**)	2
(1, 1, 1)	(**1**, **1**, **1**, **1**, **0**)	4

The syndrome decoding problem

$c' = c + e,\ \omega(e) \leqslant t$

The syndrome decoding problem

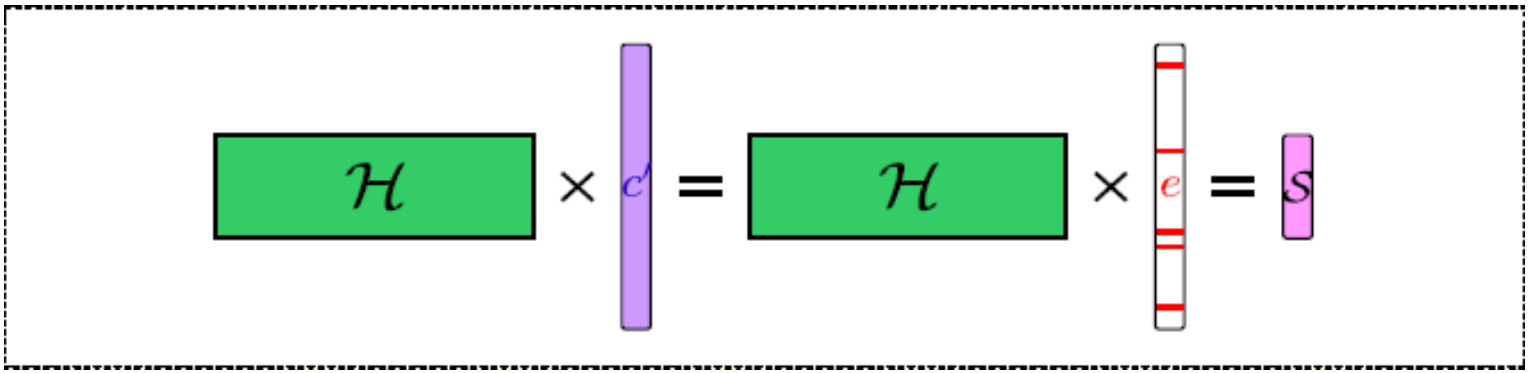
$c' = c + e,\ \omega(e) \leqslant t$

$$\forall c = (c_1, \ldots, c_n) \in \mathbb{F}_2^n,\ c \in \mathcal{C} \Leftrightarrow H^t c = 0$$

The syndrome decoding problem

$c' = c + e,\ \omega(e) \leqslant t$

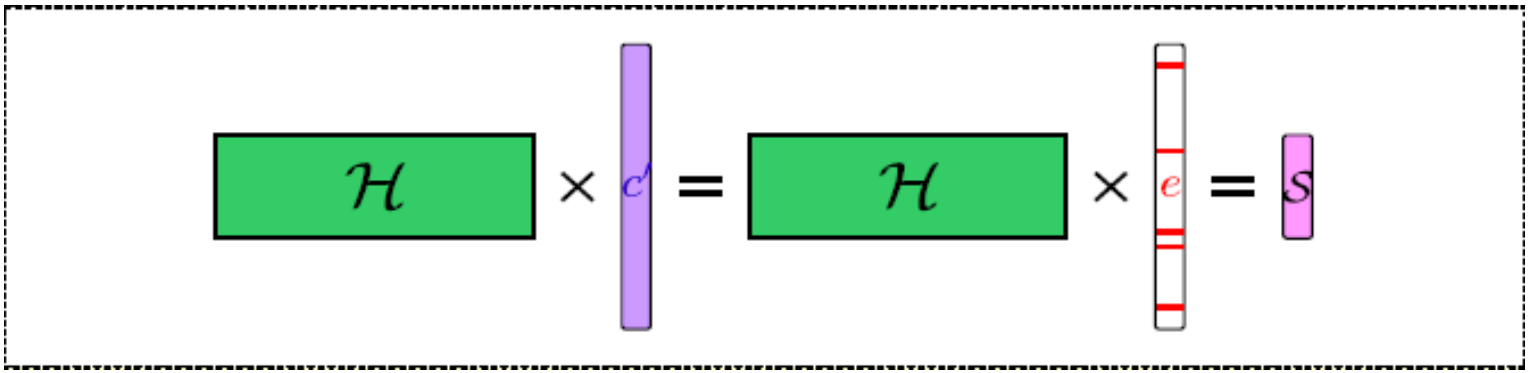
$$\forall c = (c_1, \ldots, c_n) \in \mathbb{F}_2^n,\ c \in \mathcal{C} \Leftrightarrow H^t c = 0$$



The syndrome decoding problem

$c' = c + e,\ \omega(e) \leqslant t$

$$\forall c = (c_1, \ldots, c_n) \in \mathbb{F}_2^n,\ c \in \mathcal{C} \Leftrightarrow H^t c = 0$$



Syndrome

s is called the syndrome of e.

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
H=C.parity_check_matrix()
V = VectorSpace(GF(2), 5)
for i in range(5):
    erreur = V([0]*i+[1]+[0]*(4-i))
    print(erreur, H*erreur)
```

Sagemath example

```
G = matrix(GF(2), [[1, 0, 0, 1, 0],
                   [0, 1, 0, 1, 1],
                   [0, 0, 1, 1, 1]])
C=LinearCode(G)
H=C.parity_check_matrix()
V = VectorSpace(GF(2), 5)
for i in range(5):
    erreur = V([0]*i+[1]+[0]*(4-i))
    print(erreur, H*erreur)
```

$$
\begin{array}{ll}
(1, 0, 0, 0, 0) & (1, 0) \\
(0, 1, 0, 0, 0) & (0, 1) \\
(0, 0, 1, 0, 0) & (0, 1) \\
(0, 0, 0, 1, 0) & (1, 0) \\
(0, 0, 0, 0, 1) & (1, 1)
\end{array}
$$

Syndrome decoding of Hamming Code

Parameters of Hamming Code :

- **Length (n)** : $2^r - 1$,
- **Dimension (k)** : $2^r - 1 - r$, hence r is the number of redundancy bits.
- **Minimum Distance ($d_{\min}$)** : Always 3, enabling single error correction.

Parity-Check Matrix (H) :

- All non-zero vectors of $GF(2^{n-k})$ appear as columns of H.
- Ensures each syndrome uniquely identifies a single-bit error.

Hamming(7,4) Code Example

```
C = codes.HammingCode(GF(2), 3)
print(C.generator_matrix())
print(C.parity_check_matrix())
```

Generator Matrix (G) :

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Parity-Check Matrix (H) :

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Syndrome Table for Hamming(7,4)

Syndrome Table for Single-Bit Errors :

Error	Syndrome (s)
$(1,0,0,0,0,0,0)$	$(1,0,0)$
$(0,1,0,0,0,0,0)$	$(0,1,0)$
$(0,0,1,0,0,0,0)$	$(1,1,0)$
$(0,0,0,1,0,0,0)$	$(0,0,1)$
$(0,0,0,0,1,0,0)$	$(1,0,1)$
$(0,0,0,0,0,1,0)$	$(0,1,1)$
$(0,0,0,0,0,0,1)$	$(1,1,1)$

Encoding and Received Word Example

Message (m) : $m = (1, 0, 1, 1)$

Codeword (c) :

$$c = m \cdot G = (1, 0, 1, 1) \cdot G = (1, 0, 1, 1, 1, 0, 0)$$

Received Word (y) :

$$y = c \text{ with error at position } 5 : y = (1, 0, 1, 1, 0, 0, 0)$$

Syndrome Decoding

Syndrome of Received Word (s) :

$$s = H \cdot {}^t y = (1, 0, 1)$$

Syndrome Decoding

Syndrome of Received Word (s) :

$$s = H \cdot {}^t y = (1, 0, 1)$$

Syndrome Table :

Error	Syndrome (s)
$(1,0,0,0,0,0,0)$	$(1,0,0)$
$(0,1,0,0,0,0,0)$	$(0,1,0)$
$(0,0,1,0,0,0,0)$	$(1,1,0)$
$(0,0,0,1,0,0,0)$	$(0,0,1)$
$(0,0,0,0,1,0,0)$	$(1,0,1)$
$(0,0,0,0,0,1,0)$	$(0,1,1)$
$(0,0,0,0,0,0,1)$	$(1,1,1)$

Syndrome Decoding

Syndrome of Received Word (s) :

$$s = H \cdot {}^{t}y = (1, 0, 1)$$

Syndrome Table :

Error	Syndrome (s)
$(1, 0, 0, 0, 0, 0, 0)$	$(1, 0, 0)$
$(0, 1, 0, 0, 0, 0, 0)$	$(0, 1, 0)$
$(0, 0, 1, 0, 0, 0, 0)$	$(1, 1, 0)$
$(0, 0, 0, 1, 0, 0, 0)$	$(0, 0, 1)$
$(0, 0, 0, 0, 1, 0, 0)$	$(1, 0, 1)$
$(0, 0, 0, 0, 0, 1, 0)$	$(0, 1, 1)$
$(0, 0, 0, 0, 0, 0, 1)$	$(1, 1, 1)$

Original codeword : (1, 0, 1, 1, 0, 0, 0)$\oplus$ (0,0,0,0,1,0,0) = (1, 0, 1, 1, 1, 0, 0)

Syndrome Decoding

Syndrome of Received Word (s) :

$$s = H \cdot {}^{t}y = (1, 0, 1)$$

Syndrome Table :

Error	Syndrome (s)
$(1, 0, 0, 0, 0, 0, 0)$	$(1, 0, 0)$
$(0, 1, 0, 0, 0, 0, 0)$	$(0, 1, 0)$
$(0, 0, 1, 0, 0, 0, 0)$	$(1, 1, 0)$
$(0, 0, 0, 1, 0, 0, 0)$	$(0, 0, 1)$
$(0, 0, 0, 0, 1, 0, 0)$	$(1, 0, 1)$
$(0, 0, 0, 0, 0, 1, 0)$	$(0, 1, 1)$
$(0, 0, 0, 0, 0, 0, 1)$	$(1, 1, 1)$

Original codeword : (1, 0, 1, 1, 0, 0, 0)$\oplus$ (0,0,0,0,1,0,0) = (1, 0, 1, 1, 1, 0, 0)

Original message : Extract the first four bits from (**1, 0, 1, 1**, 1, 0, 0) $\rightarrow$ (1,0,1,1)

Minimum Hamming Distance and Minimum Weight

Definition :

- Let $\mathbf{x}, \mathbf{y} \in GF(2)^n$.
- The **Hamming distance** between $\mathbf{x}$ and $\mathbf{y}$ is defined as :

$$d_H(\mathbf{x}, \mathbf{y}) = \text{number of positions where } x_i \neq y_i.$$

Minimum Hamming Distance and Minimum Weight

Definition :

- Let $\mathbf{x}, \mathbf{y} \in GF(2)^n$.
- The **Hamming distance** between $\mathbf{x}$ and $\mathbf{y}$ is defined as :

$$d_H(\mathbf{x}, \mathbf{y}) = \text{number of positions where } x_i \neq y_i.$$

Example :

- Let $\mathbf{x} = (1, 0, 1, 1, 0)$ and $\mathbf{y} = (1, 1, 0, 1, 0)$ in $GF(2)^5$.
- $d_H(\mathbf{x}, \mathbf{y}) = 2$.

Minimum Hamming Distance and Minimum Weight

Definition :

- Let $\mathbf{x}, \mathbf{y} \in GF(2)^n$.
- The **Hamming distance** between $\mathbf{x}$ and $\mathbf{y}$ is defined as :

$$d_H(\mathbf{x}, \mathbf{y}) = \text{number of positions where } x_i \neq y_i.$$

Example :

- Let $\mathbf{x} = (1, 0, 1, 1, 0)$ and $\mathbf{y} = (1, 1, 0, 1, 0)$ in $GF(2)^5$.
- $d_H(\mathbf{x}, \mathbf{y}) = 2$.

Minimum Hamming Distance ($d_{\min}$) :

- The minimum Hamming distance of a code is the smallest number of bit positions in which two distinct codewords differ.
- Mathematically, $d_{\min} = \min_{c_i, c_j \in C, i \neq j} d_H(c_i, c_j)$, where C is the set of codewords.

Minimum Hamming Distance and Minimum Weight

Definition :

- Let $\mathbf{x}, \mathbf{y} \in GF(2)^n$.
- The **Hamming distance** between $\mathbf{x}$ and $\mathbf{y}$ is defined as :

$$d_H(\mathbf{x}, \mathbf{y}) = \text{number of positions where } x_i \neq y_i.$$

Example :

- Let $\mathbf{x} = (1, 0, 1, 1, 0)$ and $\mathbf{y} = (1, 1, 0, 1, 0)$ in $GF(2)^5$.
- $d_H(\mathbf{x}, \mathbf{y}) = 2$.

Minimum Hamming Distance ($d_{\min}$) :

- The minimum Hamming distance of a code is the smallest number of bit positions in which two distinct codewords differ.
- Mathematically, $d_{\min} = \min_{c_i, c_j \in C, i \neq j} d_H(c_i, c_j)$, where C is the set of codewords.

Minimum Weight :

- The minimum weight of a code is the smallest number of non-zero bits in any non-zero codeword.
- For linear codes, the minimum Hamming distance is equal to the minimum weight.

Minimum Hamming Distance and Minimum Weight

Definition :

- Let $\mathbf{x}, \mathbf{y} \in GF(2)^n$.
- The **Hamming distance** between $\mathbf{x}$ and $\mathbf{y}$ is defined as :

$$d_H(\mathbf{x}, \mathbf{y}) = \text{number of positions where } x_i \neq y_i.$$

Example :

- Let $\mathbf{x} = (1, 0, 1, 1, 0)$ and $\mathbf{y} = (1, 1, 0, 1, 0)$ in $GF(2)^5$.
- $d_H(\mathbf{x}, \mathbf{y}) = 2$.

Minimum Hamming Distance ($d_{\min}$) :

- The minimum Hamming distance of a code is the smallest number of bit positions in which two distinct codewords differ.
- Mathematically, $d_{\min} = \min_{c_i, c_j \in C, i \neq j} d_H(c_i, c_j)$, where C is the set of codewords.

Minimum Weight :

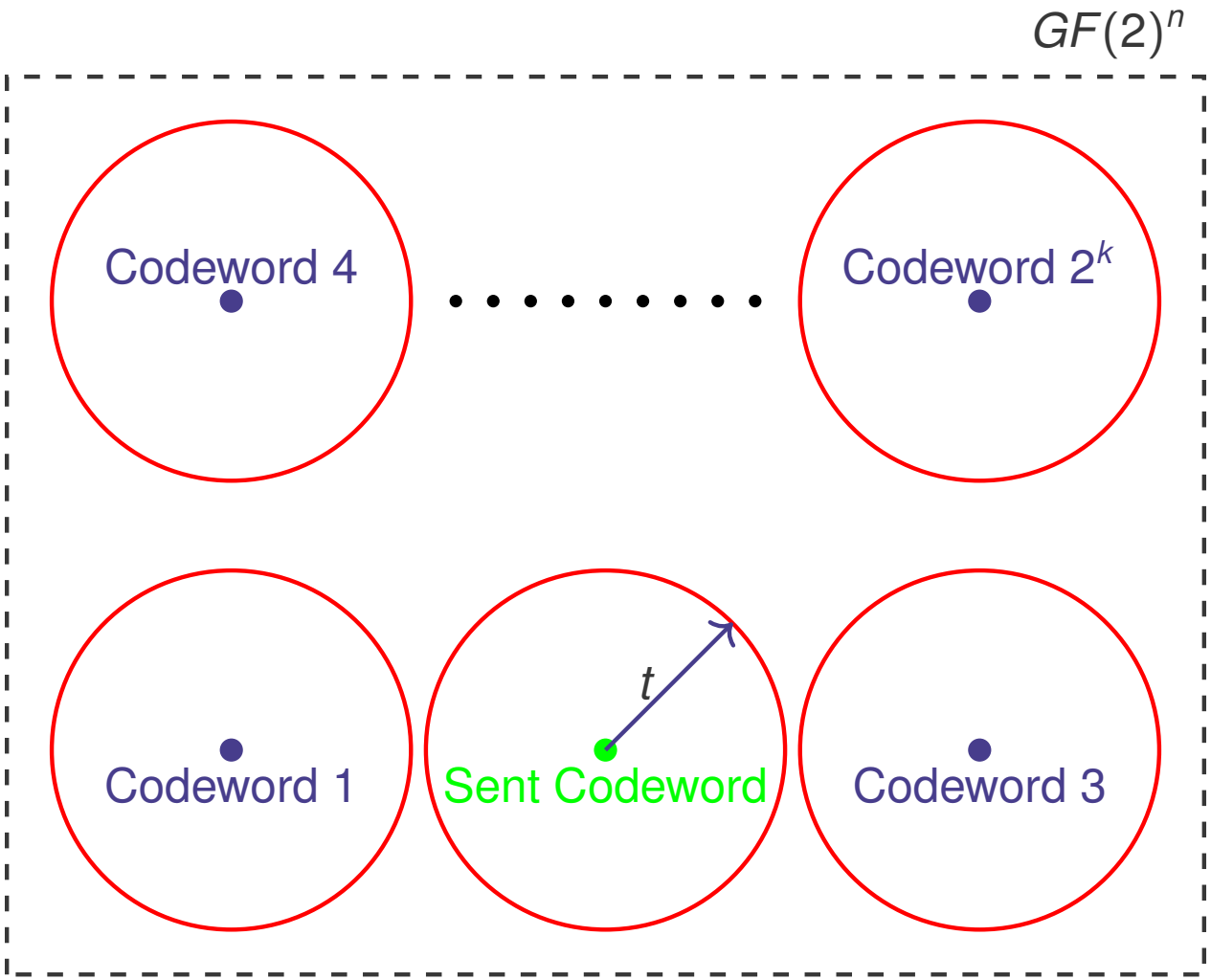
- The minimum weight of a code is the smallest number of non-zero bits in any non-zero codeword.
- For linear codes, the minimum Hamming distance is equal to the minimum weight.

Error-Correcting Capacity :

- A code with a minimum distance $d_{\min}$ can correct up to $\lfloor \frac{d_{\min}-1}{2} \rfloor$ errors.

Error-Correcting Capacity and Spheres

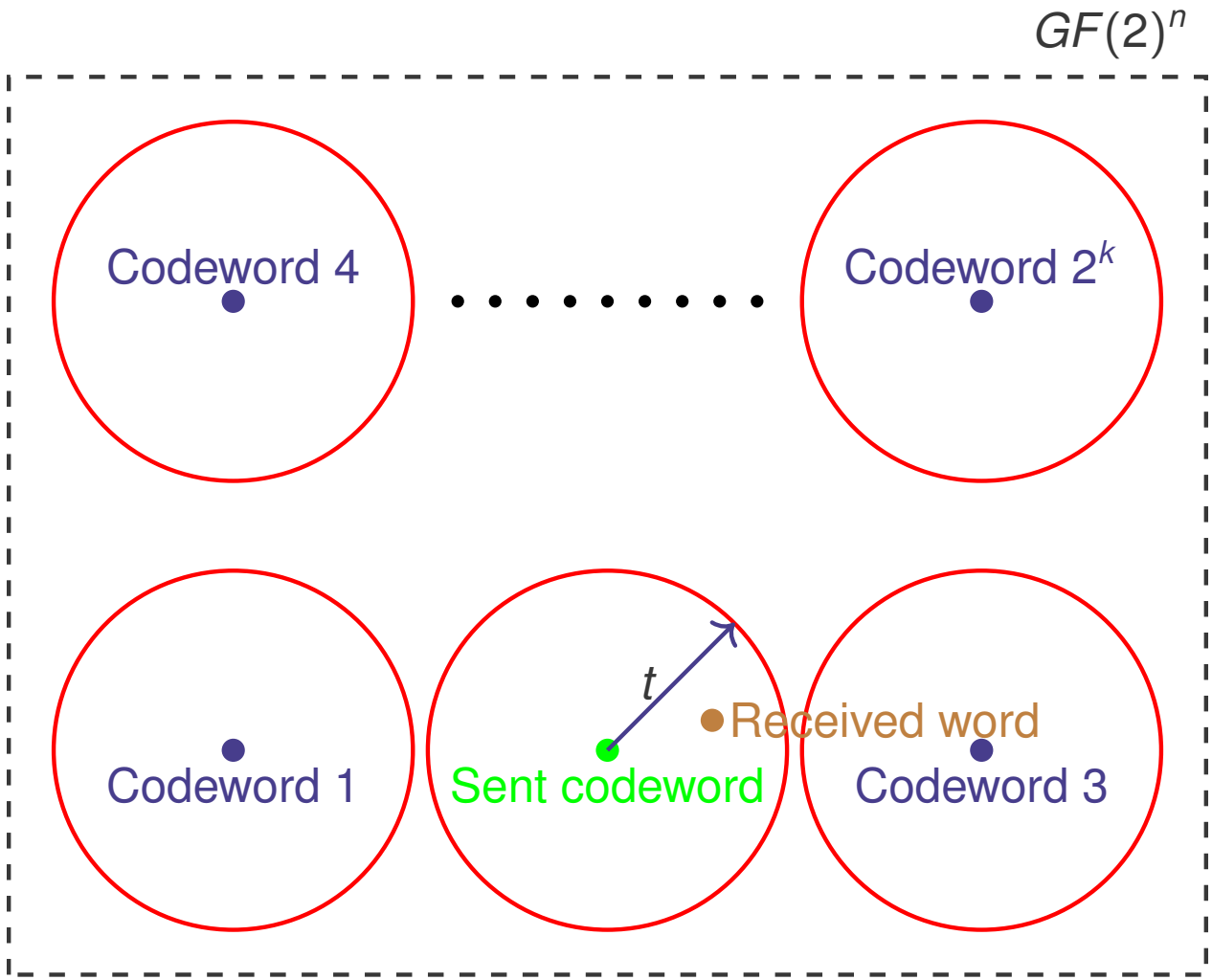
- Each codeword is in $GF(2)^n$. Draw a sphere centered on each codeword.
- Radius of each sphere : $t = \lfloor \frac{d_{\min}-1}{2} \rfloor$ (hence spheres does not intersect).



Decodable Case - Received Vector Inside a Sphere

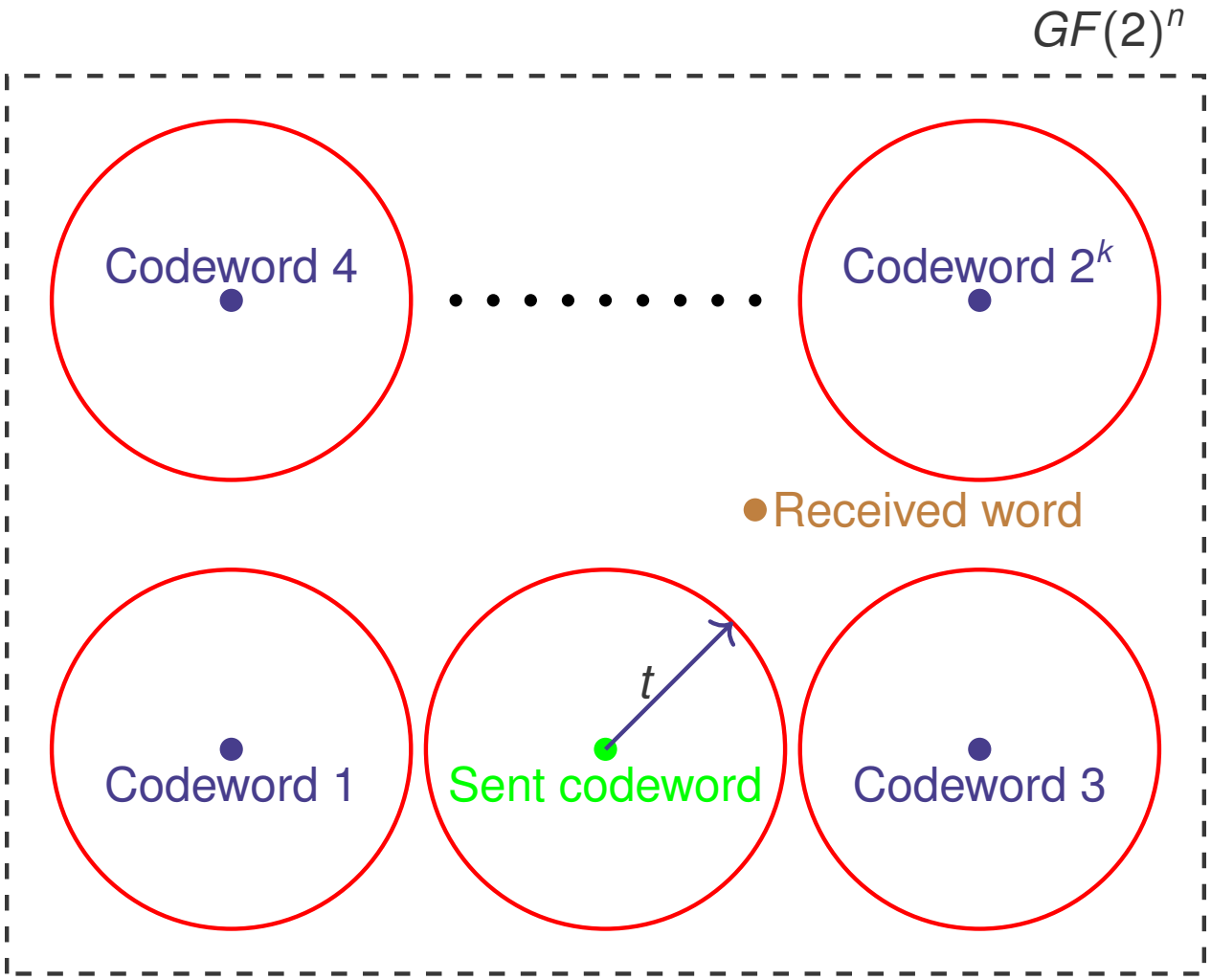
Unique Decoding :

- If no more than t errors occur in the sent codeword, then the received vector falls within the sphere centered on the sent codeword and can be uniquely decoded



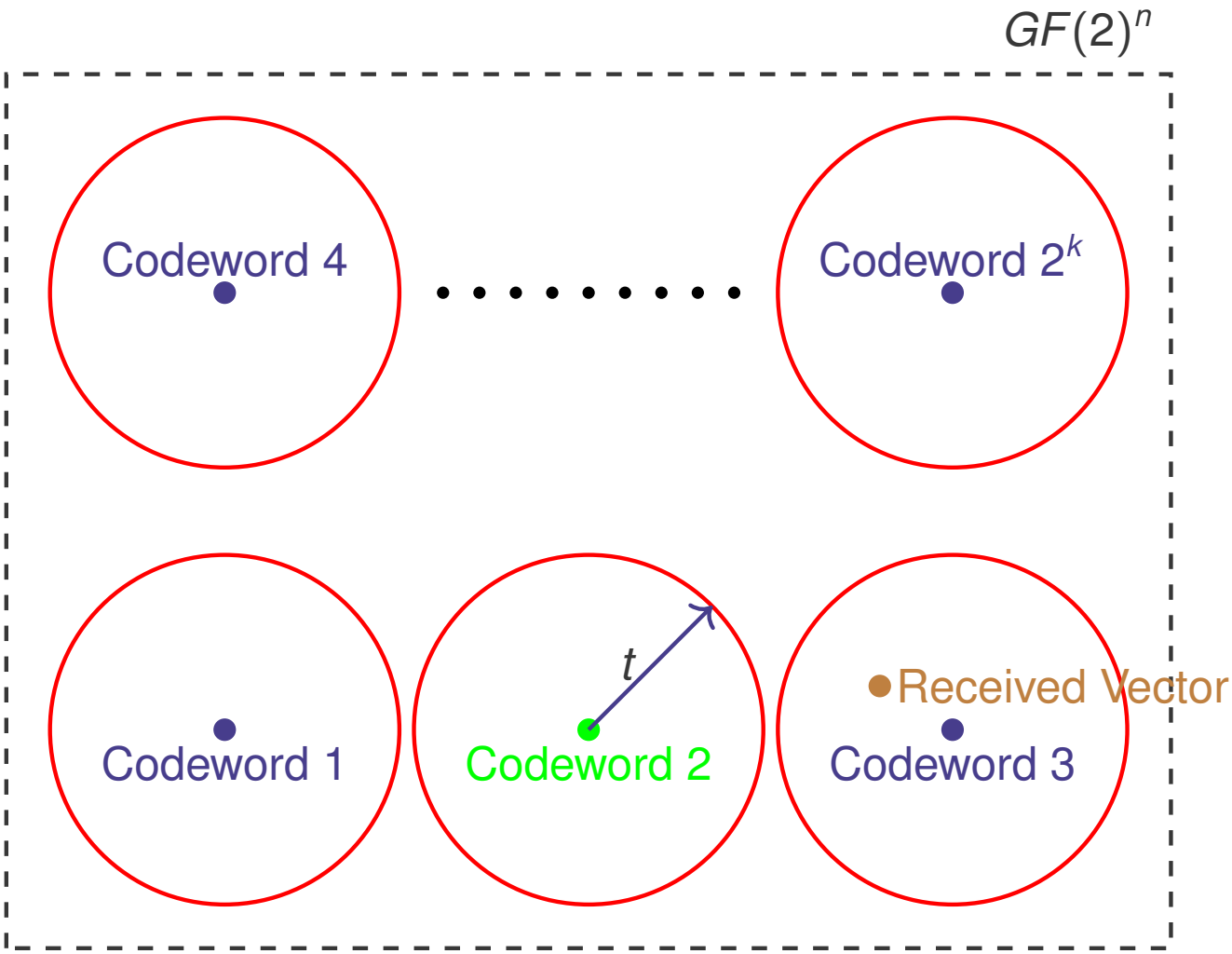
Decoding failure

- If the received vector falls does not lie within any sphere, it cannot be decoded.



Decoding Error - Received Vector in Another Sphere

- If the received vector falls into the wrong sphere, it will be incorrectly decoded.



The syndrome decoding problem

Let $\mathcal{C}[n, k, t]$ a code, and $c' \in \mathbb{F}_2^n$, can we decode c' ?

The syndrome decoding problem

Let $\mathcal{C}[n, k, t]$ a code, and $c' \in \mathbb{F}_2^n$, can we decode c' ?

Can we find e, such that $c' = c + e$, $c \in \mathcal{C}$, $\omega(e) \leqslant t$?

The syndrome decoding problem

Let $\mathcal{C}[n, k, t]$ a code, and $c' \in \mathbb{F}_2^n$, can we decode c' ?

Can we find e, such that $c' = c + e$, $c \in \mathcal{C}$, $\omega(e) \leqslant t$?

Let $s = H^t c' = H^t e$

The syndrome decoding problem

Let $\mathcal{C}[n, k, t]$ a code, and $c' \in \mathbb{F}_2^n$, can we decode c' ?

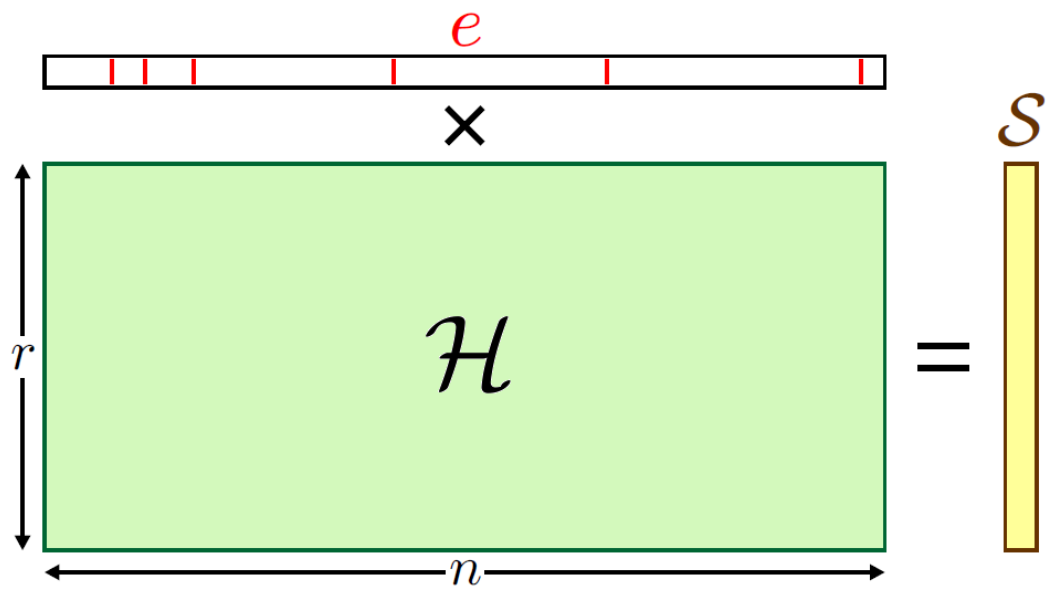
Can we find e, such that $c' = c + e$, $c \in \mathcal{C}$, $\omega(e) \leqslant t$?

Let $s = H^t c' = H^t e$

SD

Input : H a $r \times n$ binary matrix, t an integer, s a binary column vector of length r ;

Question : Does there exist a binary vector e of length n such that $H^t e = s$ and $\omega(e) \leqslant t$?



The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)
- Stern (1988)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)
- Stern (1988)
- Canteaut-Chabaud (1998)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)
- Stern (1988)
- Canteaut-Chabaud (1998)
- Johansson & Jönsson (2002)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)
- Stern (1988)
- Canteaut-Chabaud (1998)
- Johansson & Jönsson (2002)
- Bernstein, Lange , Peters (2011)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)
- Stern (1988)
- Canteaut-Chabaud (1998)
- Johansson & Jönsson (2002)
- Bernstein, Lange , Peters (2011)
- May, Meurer, Thomae (2011)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)
- Stern (1988)
- Canteaut-Chabaud (1998)
- Johansson & Jönsson (2002)
- Bernstein, Lange , Peters (2011)
- May, Meurer, Thomae (2011)
- Becker, Joux, May, Meurer (2012)

The syndrome decoding problem

Theorem [Berlekamp, Mc Eliece, Van Tilborg 1978]

SD is NP-complete

- Prange (1962)
- Mc Eliece (1978)
- Lee and Brickell (1988)
- Leon (1988)
- Stern (1988)
- Canteaut-Chabaud (1998)
- Johansson & Jönsson (2002)
- Bernstein, Lange , Peters (2011)
- May, Meurer, Thomae (2011)
- Becker, Joux, May, Meurer (2012)

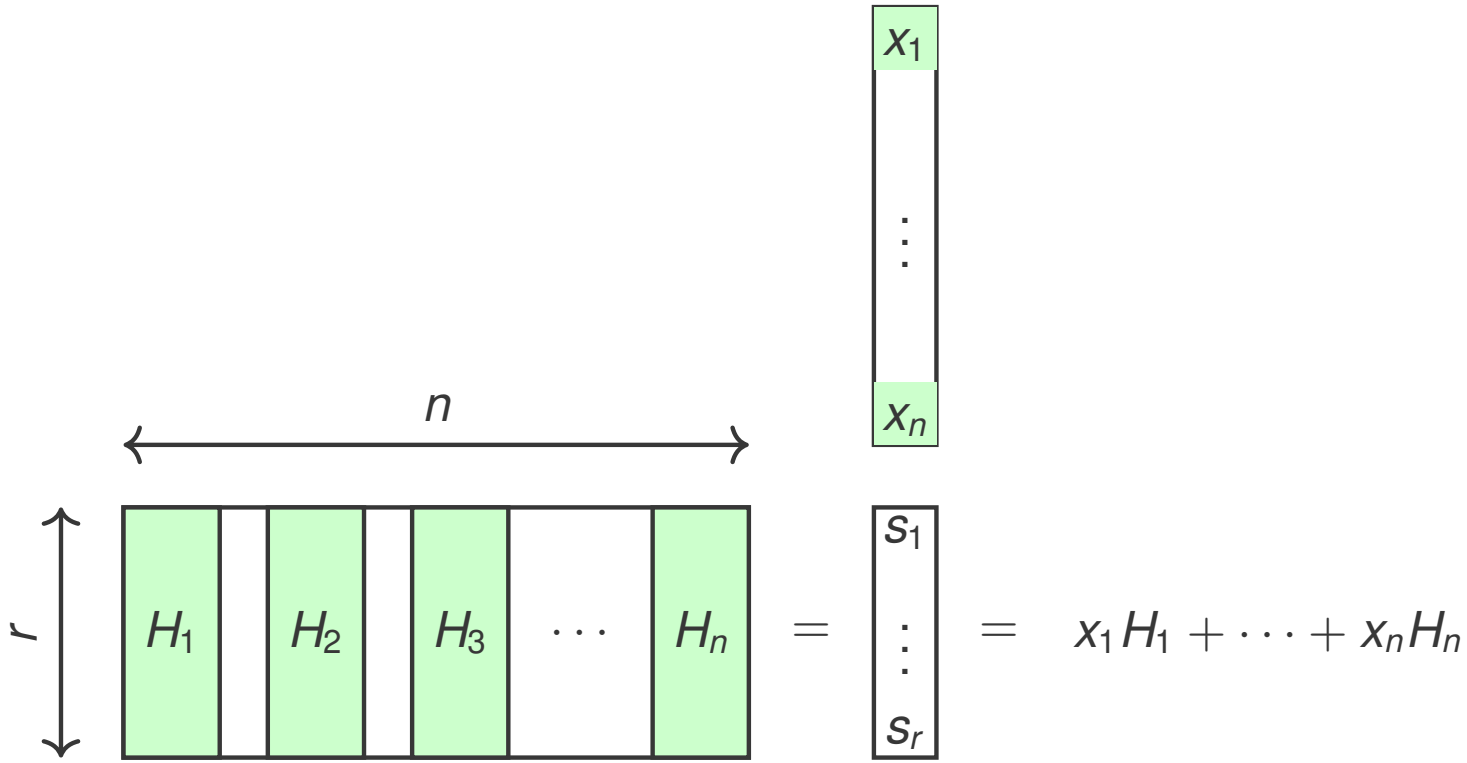
About $2^{(0.5+o(1))n/\log_2(n)}$ binary operations for a $[n, 0.5n]$ code.

Remark on Matrix-Vector Multiplication

Given a matrix H of size $r \times n$ (with r rows and n columns) and a column vector x of size n, the product Hx can be expressed as :

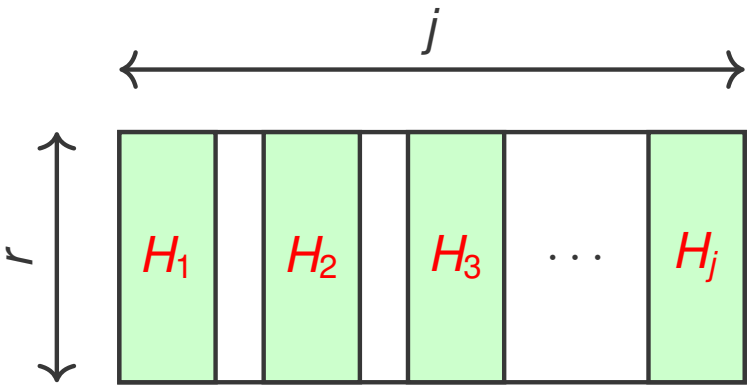
$$Hx = x_1 H_1 + x_2 H_2 + x_3 H_3 + \cdots + x_n H_n$$

where H_i represents the i-th column of H.

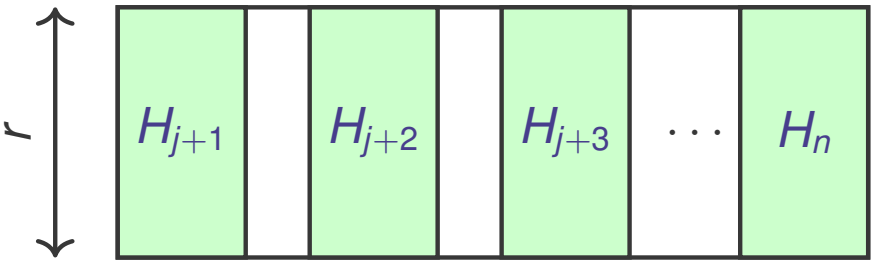


$$Hx = x_1 H_1 + x_2 H_2 + x_3 H_3 + \cdots + x_j H_j + x_{j+1} H_{j+1} + \cdots + x_n H_n$$

Let $M =$



Let $Q =$



Hence, $Hx = My + Qz$ where $y = \begin{pmatrix} x_1 \\ \vdots \\ x_j \end{pmatrix}$ and $z = \begin{pmatrix} x_{j+1} \\ \vdots \\ x_n \end{pmatrix}$.

Solving the SD problem for an arbitary code

SD

Input : H a $r \times n$ binary matrix, t an integer, s a binary column vector of length r ;

Question : Does there exist a binary vector e of length n such that $H^t e = s$ and $\omega(e) \leqslant t$?

Let P be an $n \times n$ permutation matrix then

$$H^t e = s \Leftrightarrow (HP)({}^t P^t e) = s \Leftrightarrow H' {}^t\tilde{e} = s$$

wiht $H' = HP$ and $\tilde{e} = eP$.

Solving the SD problem for an arbitary code

SD

Input : H a $r \times n$ binary matrix, t an integer, s a binary column vector of length r ;

Question : Does there exist a binary vector e of length n such that $H^t e = s$ and $\omega(e) \leqslant t$?

Let P be an $n \times n$ permutation matrix then

$$H^t e = s \Leftrightarrow (HP)({}^t P^t e) = s \Leftrightarrow H'^t \tilde{e} = s$$

wiht $H' = HP$ and $\tilde{e} = eP$.

Let U be an $r \times r$ matrix then

$$H'^t \tilde{e} = s \Leftrightarrow UH'^t \tilde{e} = Us \Leftrightarrow \tilde{H}^t \tilde{e} = \tilde{s}$$

with $\tilde{H} = UH'$ and $\tilde{s} = Us$.

Solving the SD problem for an arbitary code

SD

Input : H a $r \times n$ binary matrix, t an integer, s a binary column vector of length r ;

Question : Does there exist a binary vector e of length n such that $H^t e = s$ and $\omega(e) \leqslant t$?

Let P be an $n \times n$ permutation matrix then

$$H^t e = s \Leftrightarrow (HP)({}^t P^t e) = s \Leftrightarrow H'^t \tilde{e} = s$$

wiht $H' = HP$ and $\tilde{e} = eP$.

Let U be an $r \times r$ matrix then

$$H'^t \tilde{e} = s \Leftrightarrow UH'^t \tilde{e} = Us \Leftrightarrow \tilde{H}^t \tilde{e} = \tilde{s}$$

with $\tilde{H} = UH'$ and $\tilde{s} = Us$.

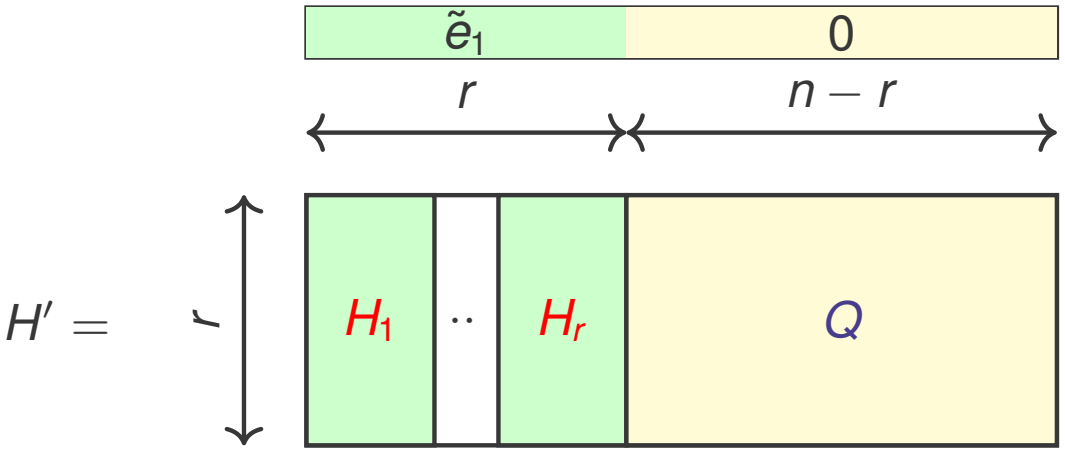
To sum up

$$H^t e = s \Leftrightarrow \tilde{H}^t \tilde{e} = \tilde{s}$$

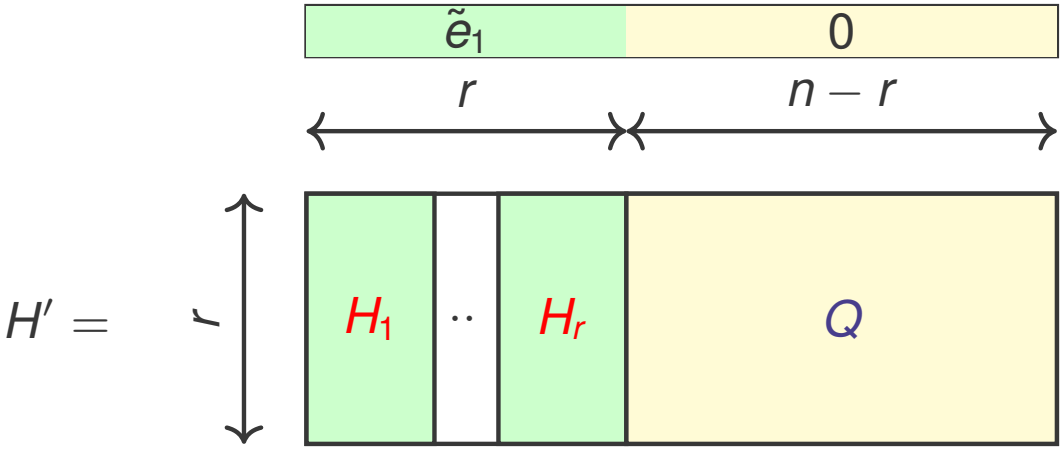
with $\tilde{H} = UHP$, $\tilde{e} = eP$ and $\tilde{s} = Us$. Notice that $\omega(e) = \omega(\tilde{e})$.

Solving the SD problem for an arbitary code

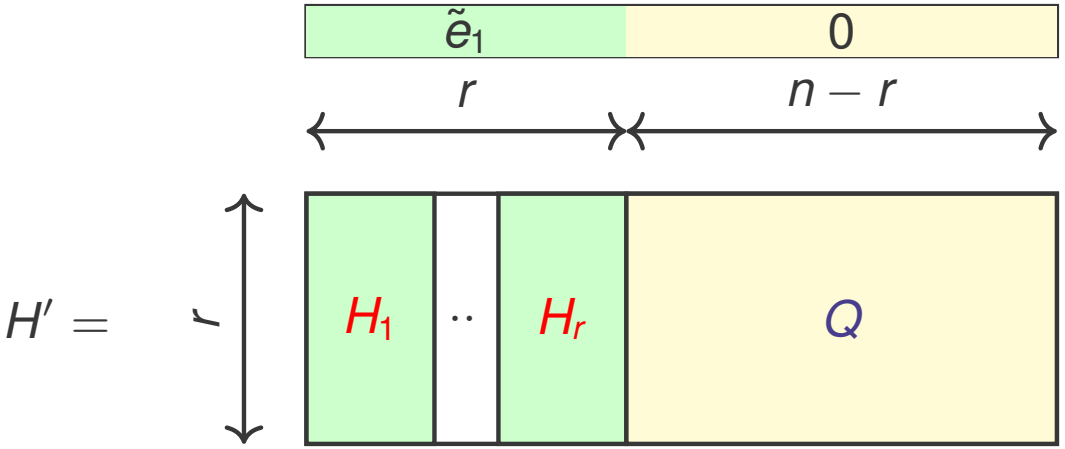
Randomly choose r columns of H and move them to the first r columns (i.e compute $H' = HP$) hoping that $\tilde{e} = eP$ satisfies :



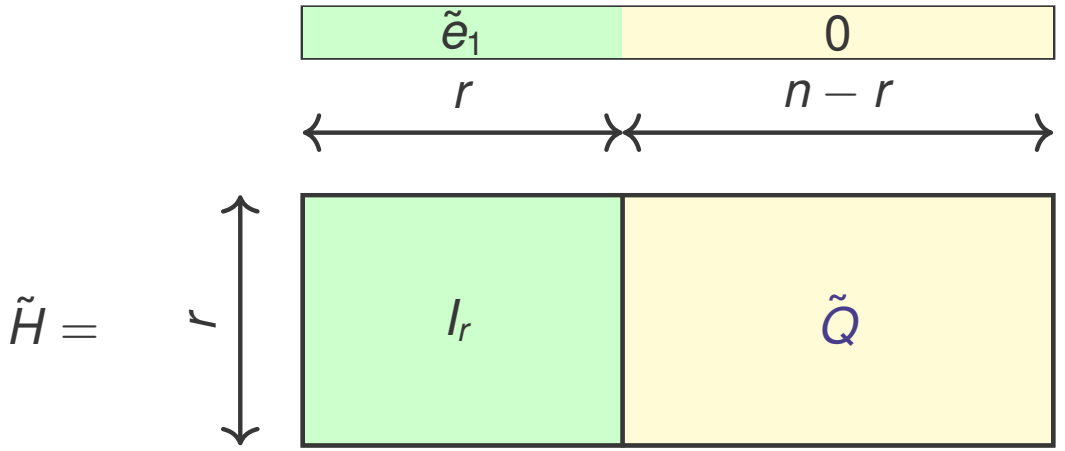
Solving the SD problem for an arbitary code



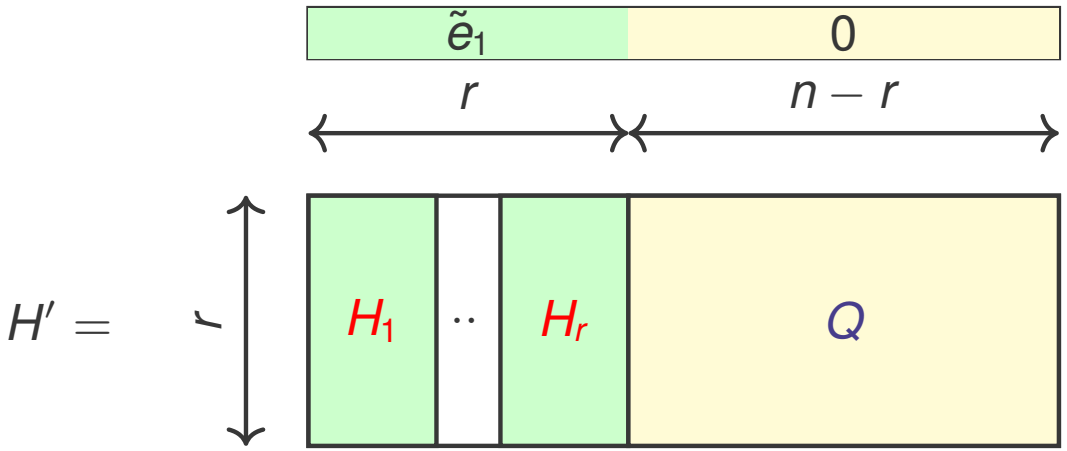
Solving the SD problem for an arbitary code



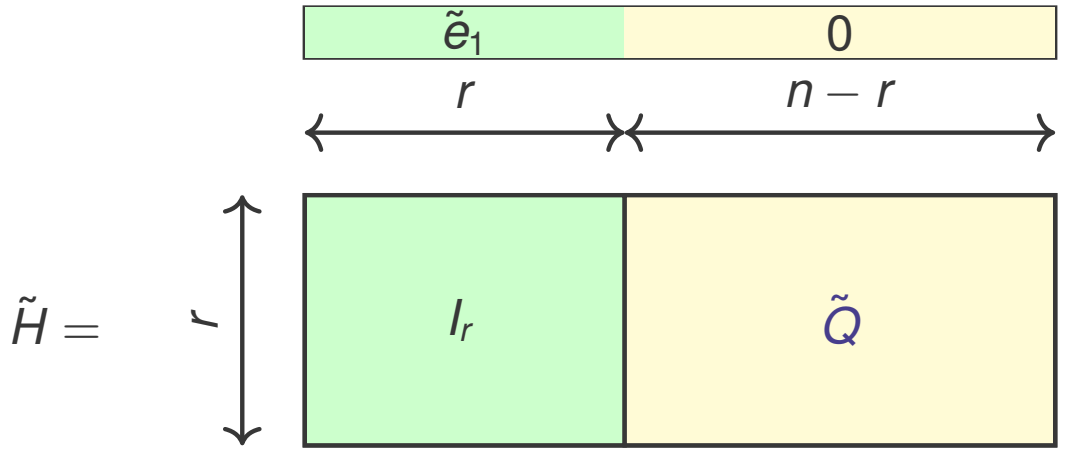
Let $M = (H_1 \dots H_r)$, if M is invertible compute $\tilde{H} = M^{-1}H$



Solving the SD problem for an arbitary code

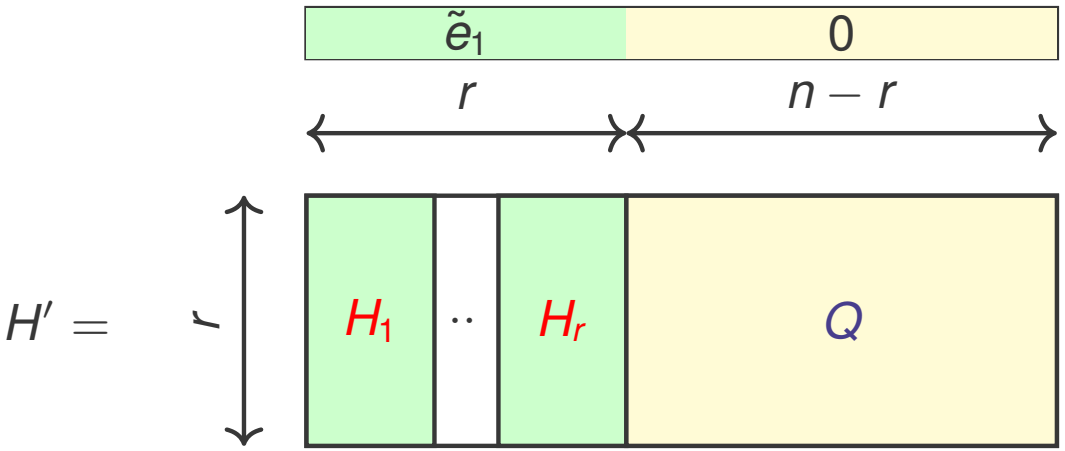


Let $M = (H_1 \dots H_r)$, if M is invertible compute $\tilde{H} = M^{-1}H$

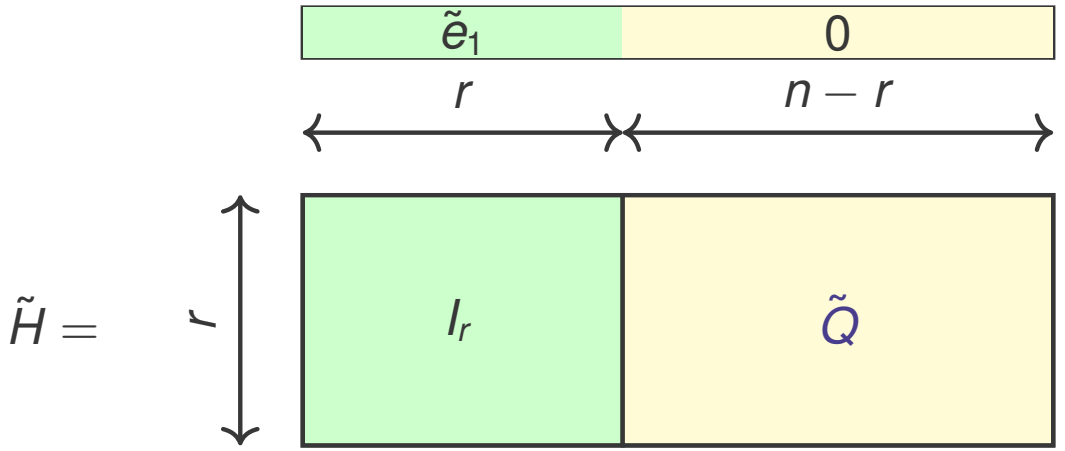


Let $\tilde{s} = M^{-1}s$, then $\tilde{H}^t\tilde{e} = \tilde{s} \Leftrightarrow {I_r}^t(\tilde{e}_1) + \tilde{Q}\begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix} = \tilde{s}$

Solving the SD problem for an arbitary code



Let $M = (H_1 \dots H_r)$, if M is invertible compute $\tilde{H} = M^{-1}H$



Let $\tilde{s} = M^{-1}s$, then $\tilde{H}^t\tilde{e} = \tilde{s} \Leftrightarrow {I_r}^t(\tilde{e}_1) + \tilde{Q}\begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix} = \tilde{s}$, that is to say $\tilde{s} = \tilde{e}_1$.

Solving the SD problem for an arbitary code

SD for an (n, k) code

Input : H a $(n-k) \times n$ binary matrix, t an integer, s a binary column vector of length r ;

Question : Does there exist a binary vector e of length n such that $H^t e = s$ and $\omega(e) \leqslant t$?

Algorithm

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- If $\omega(\tilde{s}) \leqslant t$, return $e = \tilde{e}^t P$.

until e is found.

Complexity

Algorithm

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- If $\omega(\tilde{s}) \leqslant t$, return $e = \tilde{e}^t P$.

until e is found.

- Probability to select a suitable set of $n-k$ positions : $P_{\text{good choice}} = \frac{\binom{n-k}{t}}{\binom{n}{t}}$

Complexity

Algorithm

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- If $\omega(\tilde{s}) \leqslant t$, return $e = \tilde{e}^t P$.

until e is found.

- Probability to select a suitable set of $n-k$ positions : $P_{\text{good choice}} = \frac{\binom{n-k}{t}}{\binom{n}{t}}$
- Probability that the $n-k$ first columns be invertible : $P_{\text{inv}} = (1 - \frac{1}{2^{n-k}})(1 - \frac{2}{2^{n-k}})(1 - \frac{2^2}{2^{n-k}}) \cdots (1 - \frac{2^{n-k-1}}{2^{n-k}})$

Algorithm

Repeat

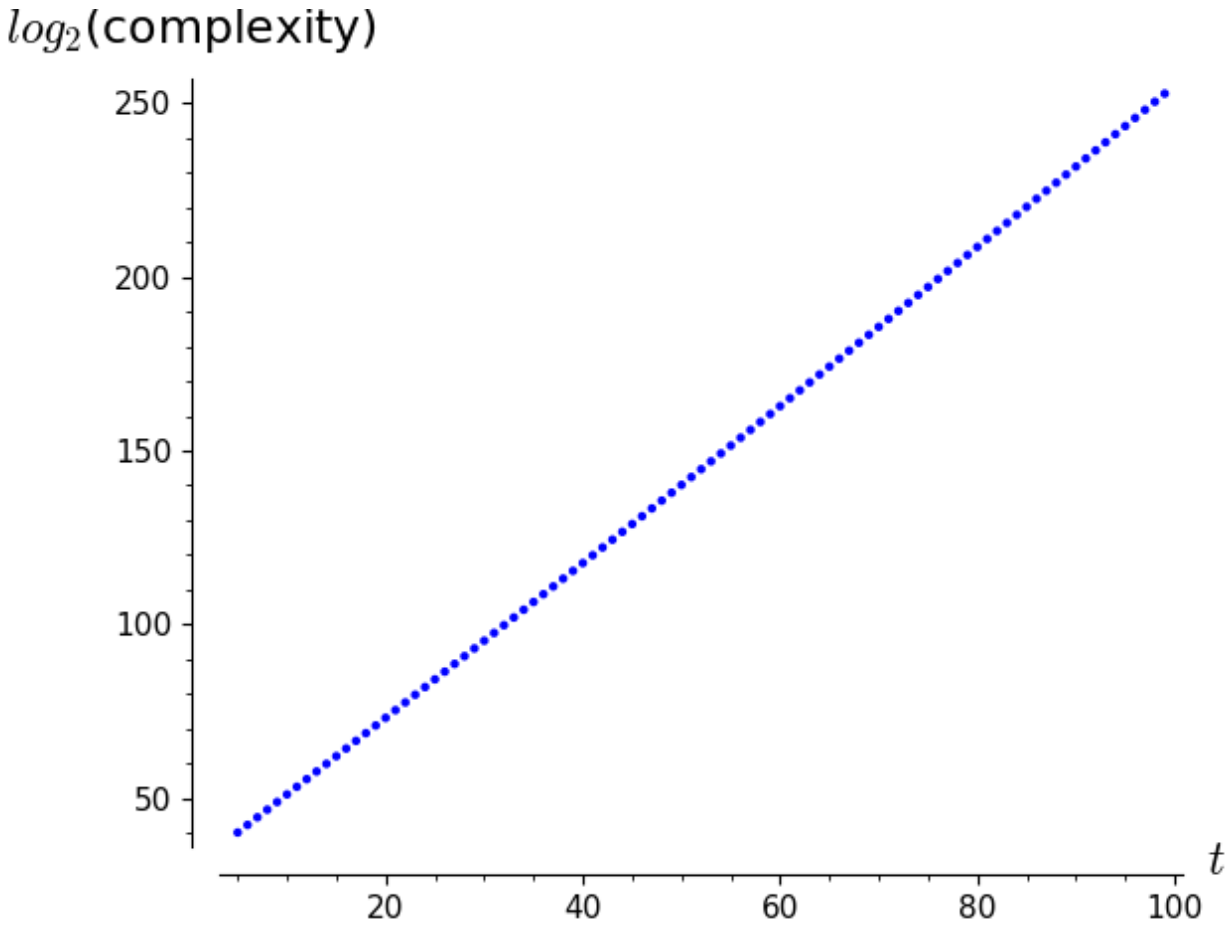
- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- If $\omega(\tilde{s}) \leqslant t$, return $e = \tilde{e}^t P$.

until e is found.

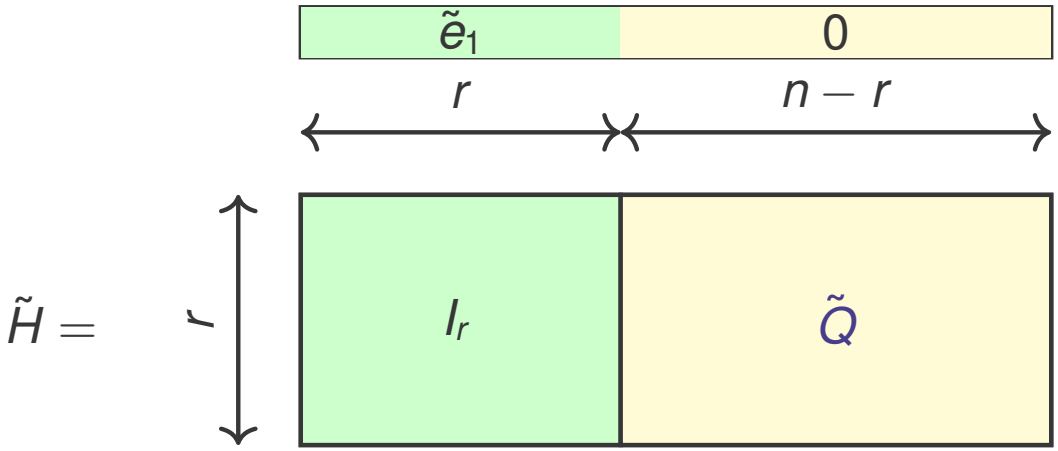
- Probability to select a suitable set of $n-k$ positions : $P_{\text{good choice}} = \frac{\binom{n-k}{t}}{\binom{n}{t}}$
- Probability that the $n-k$ first columns be invertible : $P_{\text{inv}} = (1 - \frac{1}{2^{n-k}})(1 - \frac{2}{2^{n-k}})(1 - \frac{2^2}{2^{n-k}}) \cdots (1 - \frac{2^{n-k-1}}{2^{n-k}})$
- Complexity cost : $\frac{1}{P_{\text{good choice}}} \times \frac{1}{P_{\text{inv}}} \times \mathcal{O}((n-k)^2 n)$

Complexity

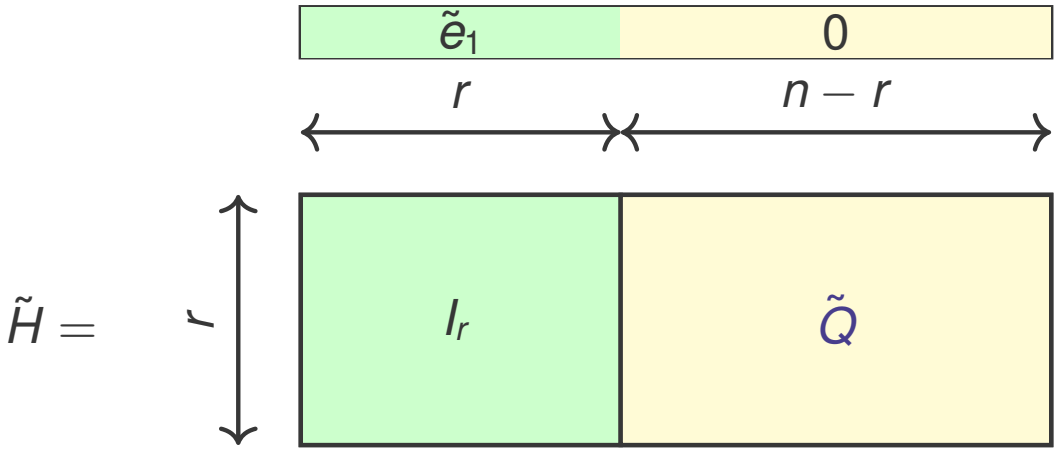
Complexity cost for a (3488,2720) code (Mc Eliece parameters)



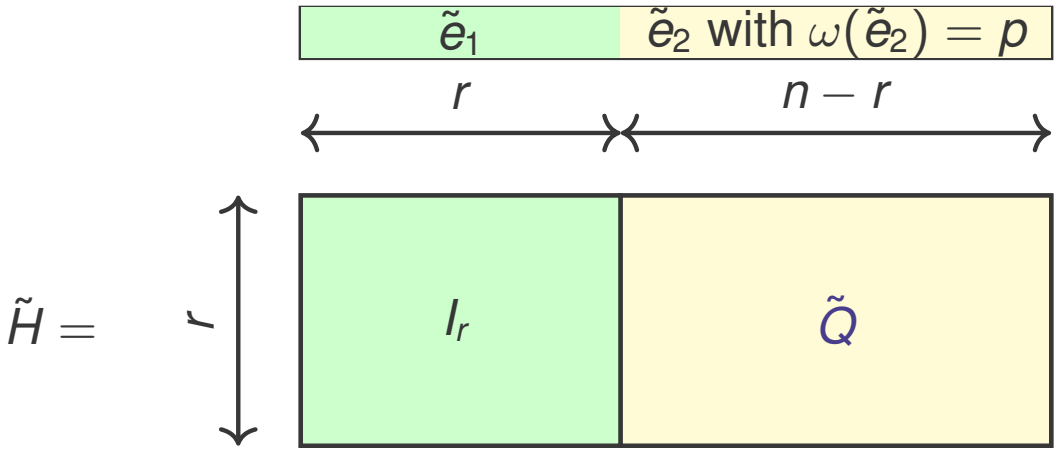
Improvement



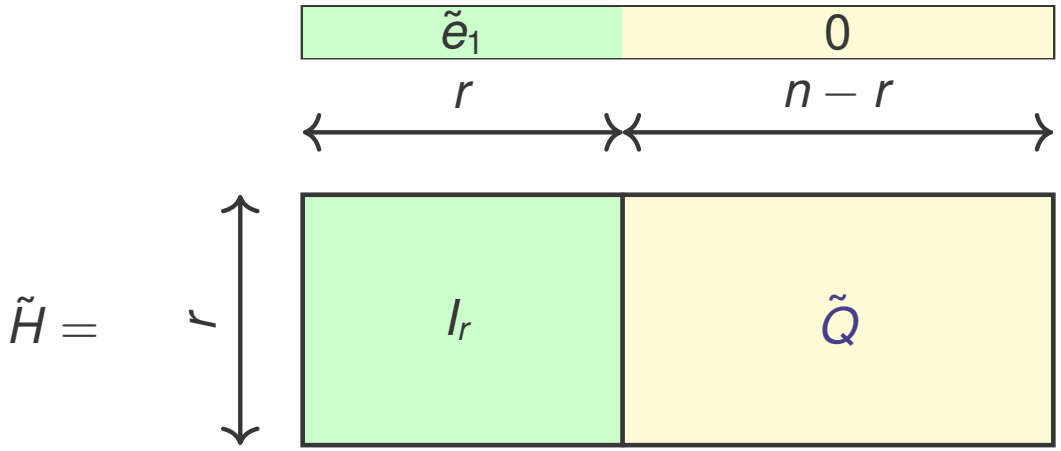
Improvement



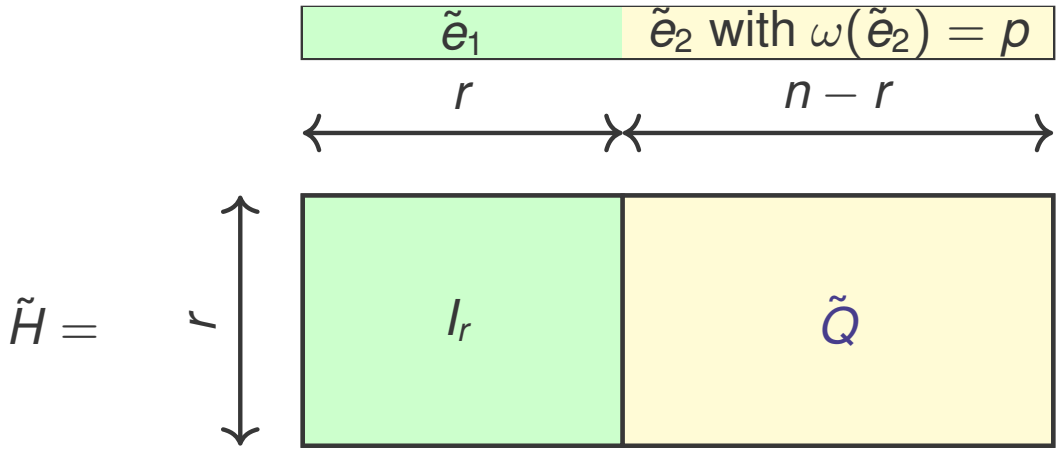
Improvement : after the swap, we hope that for p “small”



Improvement

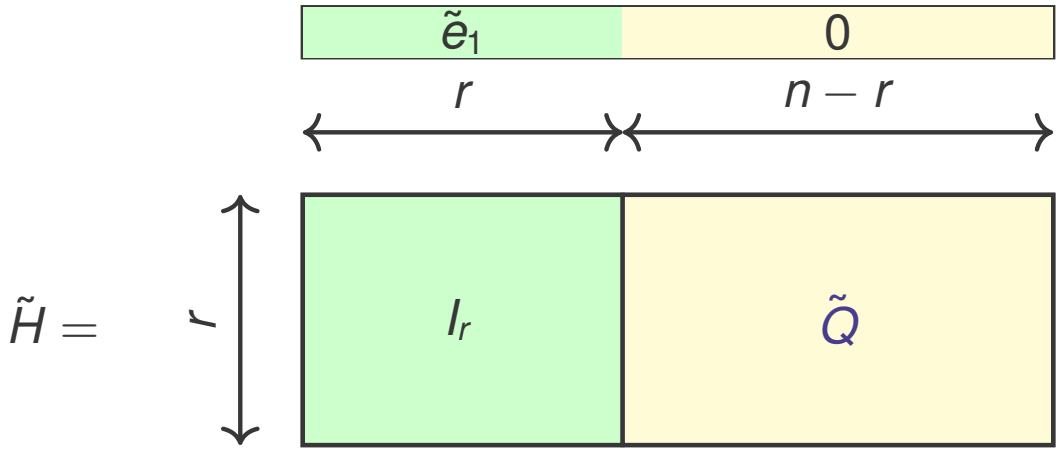


Improvement : after the swap, we hope that for p "small"

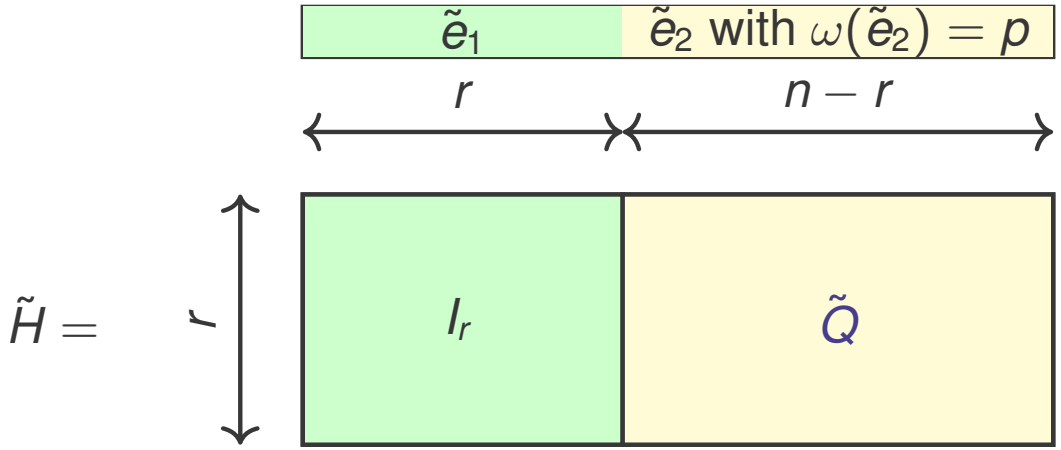


Hence $\tilde{e}_1 + \tilde{Q}\tilde{e}_2 = \tilde{s}$

Improvement



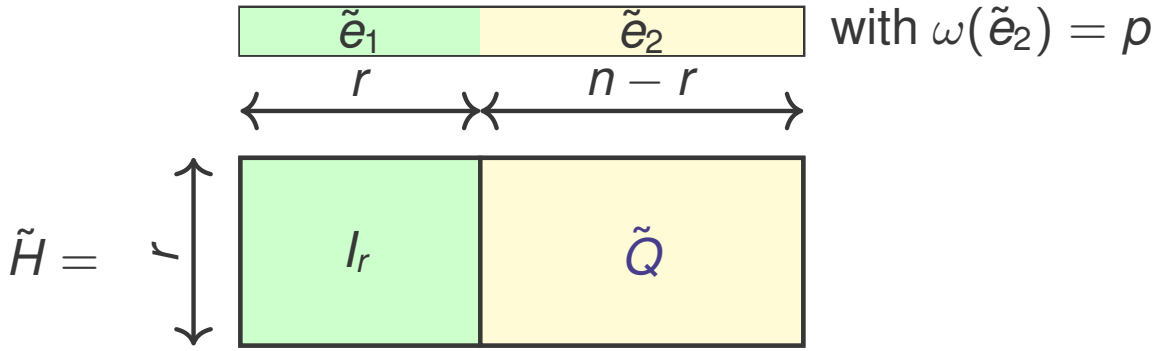
Improvement : after the swap, we hope that for p "small"



Hence $\tilde{e}_1 + \tilde{Q}\tilde{e}_2 = \tilde{s}$, which means that $\omega(\tilde{s} - \tilde{Q}\tilde{e}_2) = t - p$.

Improved Algorithm

Improvement : after the swap, we hope that for p "small"



Hence $\tilde{e}_1 + \tilde{Q}\tilde{e}_2 = \tilde{s}$, which means that $\omega(\tilde{s} - \tilde{Q}\tilde{e}_2) = t - p$.

Algorithm(p)

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- Enumerate all $\tilde{e}_2$ of weight p and compute $\tilde{s} - \tilde{Q}\tilde{e}_2$.
- If $\omega(\tilde{s} - \tilde{Q}\tilde{e}_2) \leqslant t - p$, return $e = (\tilde{e}_1 \tilde{e}_2)^t P$.

until e is found.

Algorithm(p)

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- Enumerate all $\tilde{e}_2$ of weight p and compute $\tilde{s} - \tilde{Q}\tilde{e}_2$.
- If $\omega(\tilde{s} - \tilde{Q}\tilde{e}_2) \leqslant t - p$, return $e = (\tilde{e}_1 \tilde{e}_2)^t P$.

until e is found.

- Probability to select a suitable set of $n-k$ positions : $P_{\text{good choice}} = \frac{\binom{n-k}{t-p}\binom{k}{p}}{\binom{n}{t}}$

Algorithm(p)

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- Enumerate all $\tilde{e}_2$ of weight p and compute $\tilde{s} - \tilde{Q}\tilde{e}_2$.
- If $\omega(\tilde{s} - \tilde{Q}\tilde{e}_2) \leqslant t - p$, return $e = (\tilde{e}_1 \tilde{e}_2)^t P$.

until e is found.

- Probability to select a suitable set of $n-k$ positions : $P_{\text{good choice}} = \frac{\binom{n-k}{t-p}\binom{k}{p}}{\binom{n}{t}}$
- Probability that the $n-k$ first columns be invertible :
  $P_{\text{inv}} = (1 - \frac{1}{2^{n-k}})(1 - \frac{2}{2^{n-k}})(1 - \frac{2^2}{2^{n-k}}) \cdots (1 - \frac{2^{n-k-1}}{2^{n-k}})$

Algorithm(p)

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- Enumerate all $\tilde{e}_2$ of weight p and compute $\tilde{s} - \tilde{Q}\tilde{e}_2$.
- If $\omega(\tilde{s} - \tilde{Q}\tilde{e}_2) \leqslant t - p$, return $e = (\tilde{e}_1 \tilde{e}_2)^t P$.

until e is found.

- Probability to select a suitable set of $n-k$ positions : $P_{\text{good choice}} = \frac{\binom{n-k}{t-p}\binom{k}{p}}{\binom{n}{t}}$
- Probability that the $n-k$ first columns be invertible :
  $P_{\text{inv}} = (1 - \frac{1}{2^{n-k}})(1 - \frac{2}{2^{n-k}})(1 - \frac{2^2}{2^{n-k}}) \cdots (1 - \frac{2^{n-k-1}}{2^{n-k}})$
- Enumeration of all $\tilde{e}_2$ and computation of $\tilde{s} - Q\tilde{e}_2$: $\binom{k}{p}(p(n-k) + (n-k))$.

Algorithm(p)

Repeat

- Randomly choose $n-k$ columns of H and swap them with the $n-k$ first columns. Keep a copy of the corresponding permutation P.
- Try to apply Gaussian elimination to the first $n-k$ columns to obtain a matrix in the form of $(I_{n-k} \mid Q)$.
- Apply simultenaously the same Gaussian elimination to s, to obtain $\tilde{s}$.
- Enumerate all $\tilde{e}_2$ of weight p and compute $\tilde{s} - \tilde{Q}\tilde{e}_2$.
- If $\omega(\tilde{s} - \tilde{Q}\tilde{e}_2) \leqslant t - p$, return $e = (\tilde{e}_1 \tilde{e}_2)^t P$.

until e is found.

- Probability to select a suitable set of $n-k$ positions : $P_{\text{good choice}} = \frac{\binom{n-k}{t-p}\binom{k}{p}}{\binom{n}{t}}$
- Probability that the $n-k$ first columns be invertible :
  $P_{\text{inv}} = (1 - \frac{1}{2^{n-k}})(1 - \frac{2}{2^{n-k}})(1 - \frac{2^2}{2^{n-k}}) \cdots (1 - \frac{2^{n-k-1}}{2^{n-k}})$
- Enumeration of all $\tilde{e}_2$ and computation of $\tilde{s} - Q\tilde{e}_2$: $\binom{k}{p}(p(n-k) + (n-k))$.
- Complexity cost : $\frac{1}{P_{\text{good choice}}} \times \frac{1}{P_{\text{inv}}} \times (\mathcal{O}((n-k)^2 n) + \binom{k}{p}(p(n-k) + (n-k)))$

$$\frac{1}{P_{\text{inv}}} \times \mathcal{O}((n-k)^2 n) \times \frac{\binom{n}{t}}{\binom{n-k}{t}}$$

$$\frac{1}{P_{\text{inv}}} \times \left(\mathcal{O}((n-k)^2 n) + \binom{k}{p}(p(n-k)+(n-k)) \right) \times \frac{\binom{n}{t}}{\binom{n-k}{t-p}\binom{k}{p}}$$

$$\frac{1}{P_{\text{inv}}} \times \mathcal{O}((n-k)^2 n) \times \frac{\binom{n}{t}}{\binom{n-k}{t}}$$

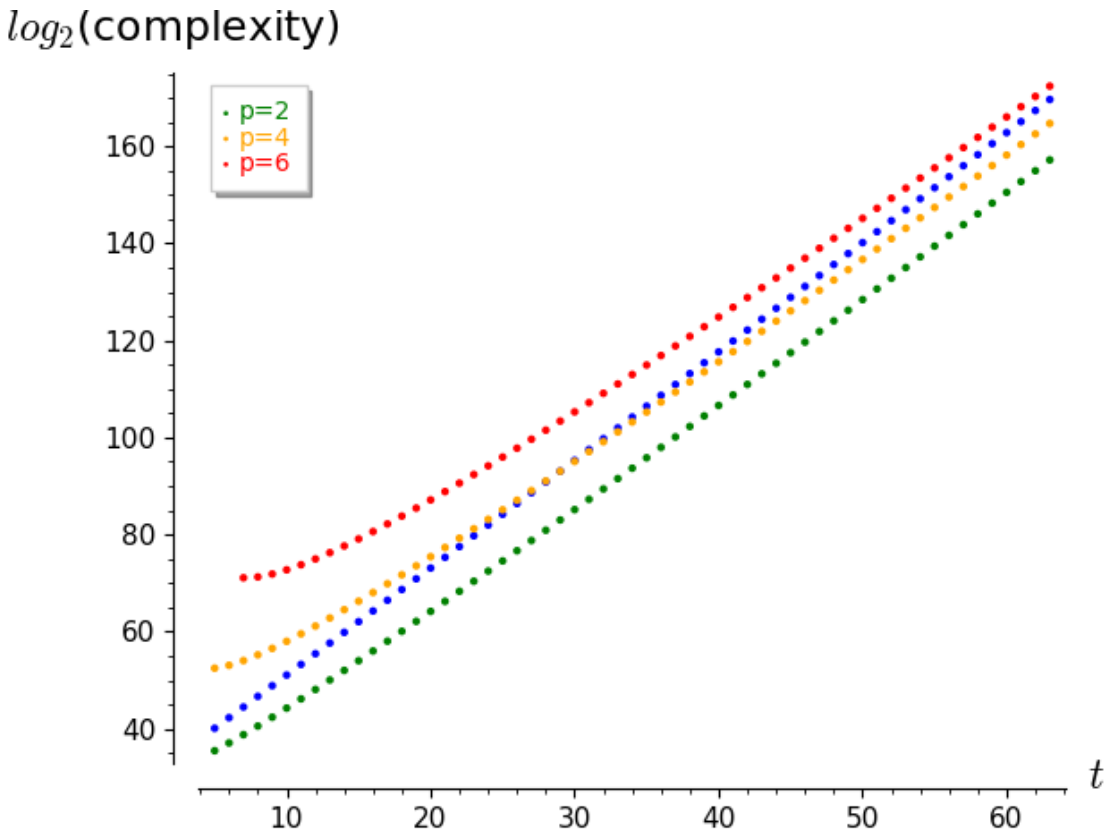
$$\frac{1}{P_{\text{inv}}} \times \left(\mathcal{O}((n-k)^2 n) + \binom{k}{p}(p(n-k)+(n-k)) \right) \times \frac{\binom{n}{t}}{\binom{n-k}{t-p}\binom{k}{p}}$$

Choose p such that enumerating words of weight p and size k has a lower computational cost than performing Gaussian elimination.

$$\frac{1}{P_{\text{inv}}} \times \mathcal{O}((n-k)^2 n) \times \frac{\binom{n}{t}}{\binom{n-k}{t}}$$

$$\frac{1}{P_{\text{inv}}} \times \left(\mathcal{O}((n-k)^2 n) + \binom{k}{p}(p(n-k)+(n-k)) \right) \times \frac{\binom{n}{t}}{\binom{n-k}{t-p}\binom{k}{p}}$$

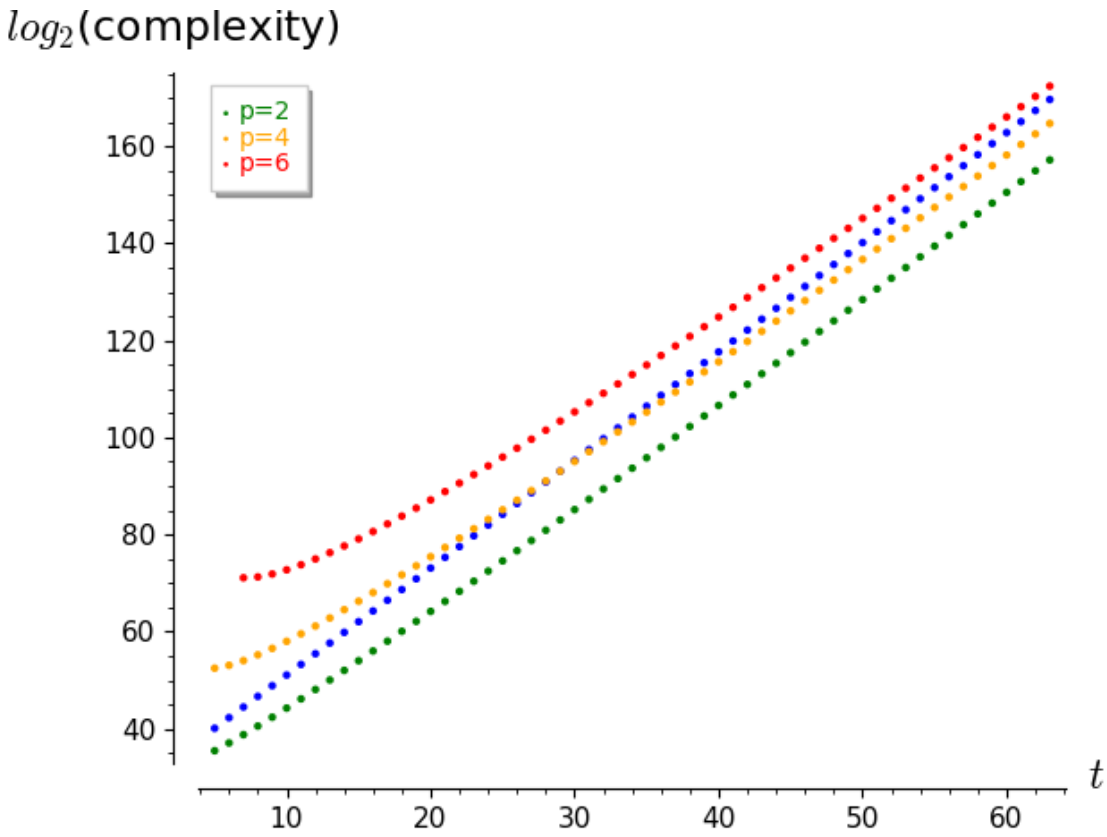
Choose p such that enumerating words of weight p and size k has a lower computational cost than performing Gaussian elimination.



$$\frac{1}{P_{\text{inv}}} \times \mathcal{O}((n-k)^2 n) \times \frac{\binom{n}{t}}{\binom{n-k}{t}}$$

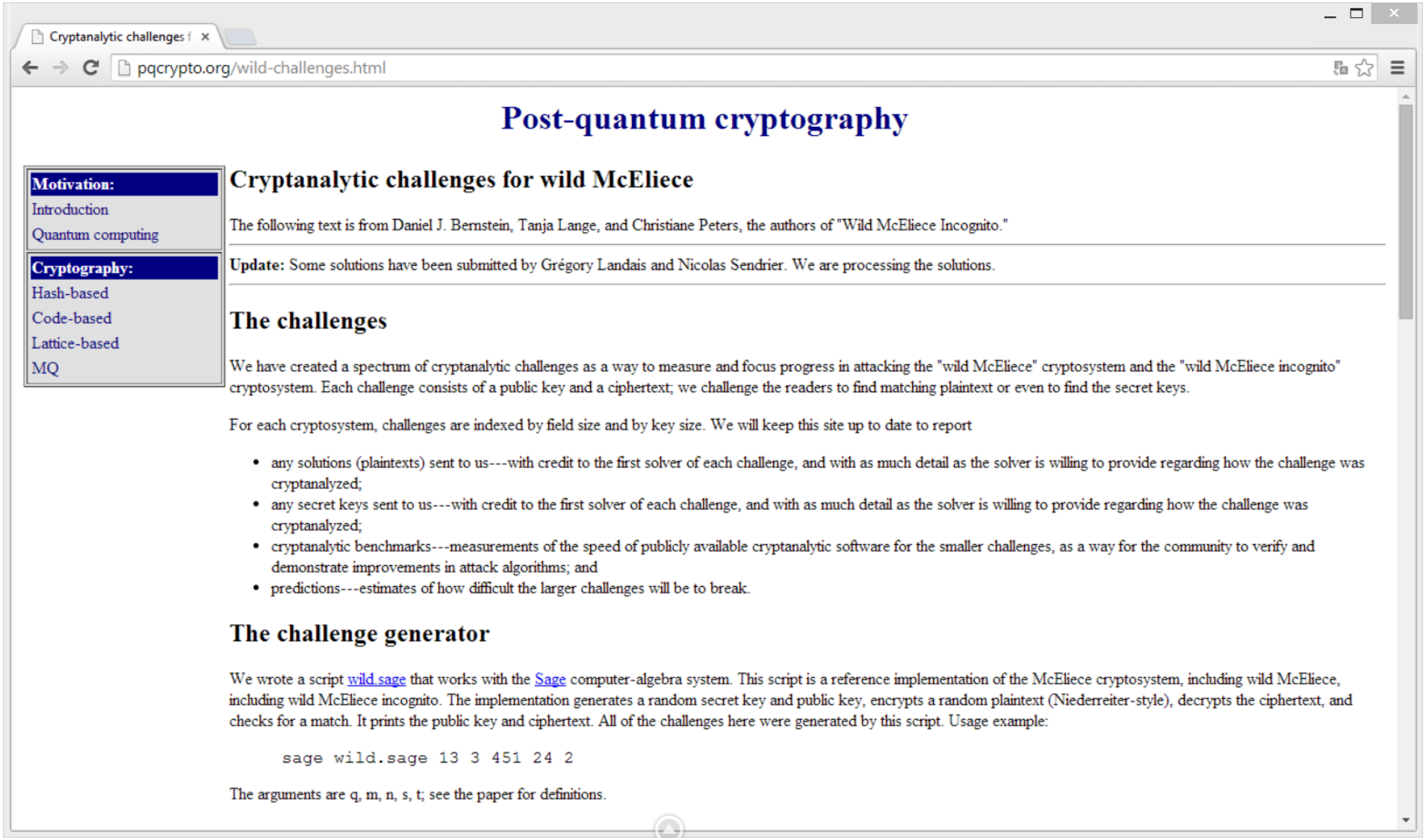
$$\frac{1}{P_{\text{inv}}} \times \left(\mathcal{O}((n-k)^2 n) + \binom{k}{p}(p(n-k)+(n-k)) \right) \times \frac{\binom{n}{t}}{\binom{n-k}{t-p}\binom{k}{p}}$$

Choose p such that enumerating words of weight p and size k has a lower computational cost than performing Gaussian elimination.



$p = 2$ is the best choice

SD Challenge



Goppa Code

To construct a Goppa code (n, t) follow these steps :

- Choose an integer m such that $2^m \geqslant n$.

Goppa Code

To construct a Goppa code (n, t) follow these steps :

- Choose an integer m such that $2^m \geqslant n$.
- Select a polynomial $g(z)$ of degree t over $GF(2^m)$.

Goppa Code

To construct a Goppa code (n, t) follow these steps :

- Choose an integer m such that $2^m \geqslant n$.
- Select a polynomial $g(z)$ of degree t over $GF(2^m)$.
- Define the set $L = \{\alpha_1, \alpha_2, \ldots, \alpha_n\}$ of elements in $GF(2^m)$ such that $g(\alpha_i) \neq 0$. So, $n = \text{card}(L)$.

Goppa Code

To construct a Goppa code (n, t) follow these steps :

- Choose an integer m such that $2^m \geqslant n$.
- Select a polynomial $g(z)$ of degree t over $GF(2^m)$.
- Define the set $L = \{\alpha_1, \alpha_2, \ldots, \alpha_n\}$ of elements in $GF(2^m)$ such that $g(\alpha_i) \neq 0$. So, $n = \text{card}(L)$.
- The Goppa code is the set of words $\mathbf{c} = (c_1, c_2, \ldots, c_n) \in GF(2)^n$ satisfying :

$$\sum_{i=1}^{n} \frac{c_i}{z - \alpha_i} \equiv 0 \mod g(z)$$

Goppa Code

To construct a Goppa code (n, t) follow these steps :

- Choose an integer m such that $2^m \geqslant n$.
- Select a polynomial $g(z)$ of degree t over $GF(2^m)$.
- Define the set $L = \{\alpha_1, \alpha_2, \ldots, \alpha_n\}$ of elements in $GF(2^m)$ such that $g(\alpha_i) \neq 0$. So, $n = \text{card}(L)$.
- The Goppa code is the set of words $\mathbf{c} = (c_1, c_2, \ldots, c_n) \in GF(2)^n$ satisfying :

$$\sum_{i=1}^{n} \frac{c_i}{z - \alpha_i} \equiv 0 \mod g(z)$$

- Dimension k satisfies $k \geqslant n - mt$, experimentally $k = n - mt$.

Goppa Code

To construct a Goppa code (n, t) follow these steps :

- Choose an integer m such that $2^m \geqslant n$.
- Select a polynomial $g(z)$ of degree t over $GF(2^m)$.
- Define the set $L = \{\alpha_1, \alpha_2, \ldots, \alpha_n\}$ of elements in $GF(2^m)$ such that $g(\alpha_i) \neq 0$. So, $n = \text{card}(L)$.
- The Goppa code is the set of words $\mathbf{c} = (c_1, c_2, \ldots, c_n) \in GF(2)^n$ satisfying :

$$\sum_{i=1}^{n} \frac{c_i}{z - \alpha_i} \equiv 0 \mod g(z)$$

- Dimension k satisfies $k \geqslant n - mt$, experimentally $k = n - mt$.
- The parity-check matrix H over $GF(2^m)$ is constructed as :

$$H = \begin{bmatrix} \frac{1}{g(\alpha_1)} & \frac{1}{g(\alpha_2)} & \cdots & \frac{1}{g(\alpha_n)} \\ \frac{\alpha_1}{g(\alpha_1)} & \frac{\alpha_2}{g(\alpha_2)} & \cdots & \frac{\alpha_n}{g(\alpha_n)} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_1^{t-1}}{g(\alpha_1)} & \frac{\alpha_2^{t-1}}{g(\alpha_2)} & \cdots & \frac{\alpha_n^{t-1}}{g(\alpha_n)} \end{bmatrix}$$

Goppa Code example

```
F.<a>=GF(2^4)
R.<z> = PolynomialRing(F)
g=z^3+z+1
t=g.degree()
L=[b for b in F.list() if g(b)!=0]
print(L)
C=codes.GoppaCode(g,L)
HH = matrix(F,[[b^i/g(b) for b in L] for i in range(t)])
print(C)
for c in C:
    print(c,HH*c)
```

```
[0, a, a^2, a^3, a + 1, a^2 + a, a^3 + a^2, a^3 + a + 1, a^2 + 1, a^3 + a, ..., a^3 + 1, 1]
[16, 4] Goppa code over GF(2)
(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0) (0, 0, 0)
(1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0) (0, 0, 0)
(0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0) (0, 0, 0)
(1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0) (0, 0, 0)
(0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1) (0, 0, 0)
(1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1) (0, 0, 0)
(0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1) (0, 0, 0)
(1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1) (0, 0, 0)
(0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1) (0, 0, 0)
(1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1) (0, 0, 0)
(0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1) (0, 0, 0)
(1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1) (0, 0, 0)
(0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0) (0, 0, 0)
(1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0) (0, 0, 0)
(0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0) (0, 0, 0)
(1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0) (0, 0, 0)
```

From $GF(2^m)$ to $GF(2)$

Every element of $GF(2^4)$ has the following form $x_0 + x_1 a + x_2 a^2 + x_3 a^3$ and can be associated to the vector $\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}$ in $GF(2)^4$.

From $GF(2^m)$ to $GF(2)$

Every element of $GF(2^4)$ has the following form $x_0 + x_1 a + x_2 a^2 + x_3 a^3$ and can be associated to the vector $\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}$ in $GF(2)^4$.

Substitute each element of H by its "binary" representation to obtain a matrix over $GF(2)$.

From $GF(2^m)$ to $GF(2)$

Every element of $GF(2^4)$ has the following form $x_0 + x_1 a + x_2 a^2 + x_3 a^3$ and can be associated to the vector $\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}$ in $GF(2)^4$.

Substitute each element of H by its "binary" representation to obtain a matrix over $GF(2)$.

$$\begin{pmatrix} 1 & a^2+1 & a & a^3+a^2+a & \cdots & 1 \\ 0 & a^3+a & a^3 & a^3+1 & \cdots & 1 \\ 0 & a^2+a+1 & a^2+a & a^2 & \cdots & 1 \end{pmatrix}$$

$$\left(\begin{array}{|c|cccccccccccccc} 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{array}\right)$$

Goppa Code generator matrix

The obtained matrix has mt rows, hence its dimension is at most mt which explains the bound on the dimension of the code : $k \geqslant n - mt$

Goppa Code generator matrix

The obtained matrix has mt rows, hence its dimension is at most mt which explains the bound on the dimension of the code : $k \geqslant n - mt$
Apply a Gaussian elimination to obtain a full rank matrix of the form $(I_{k'}|M)$

Goppa Code generator matrix

The obtained matrix has mt rows, hence its dimension is at most mt which explains the bound on the dimension of the code : $k \geqslant n - mt$
Apply a Gaussian elimination to obtain a full rank matrix of the form $(I_{k'}|M)$
Compute $G = ({}^tM|I_{n-k'})$

```
m = 4
t = 3
F.<a> = GF(2^m)
R.<z> = PolynomialRing(F)
g=z^t+z+1
L=[b for b in F.list() if g(b)!=0]
HH = matrix(F,[[b^i/g(b) for b in L] for i in range(t)])
H = matrix(GF(2),m*t,len(L))
for i in range(t):
for j in range(len(L)):
rep_bin = list(HH[i][j])
aux = 0
for ligne in range(i*m, (i+1)*m):
H[ligne,j] = rep_bin[aux]
aux += 1
H = H.echelon_form()
dimH = H.nrows()
G = transpose(H.echelon_form().matrix_from_columns(range(dimH,len(L))))
G = block_matrix([[G,matrix.identity(GF(2),len(L)-dimH)]])
```

Goppa Code generator matrix

```
G = block_matrix([[G,matrix.identity(GF(2),len(L)-dimH)]])
print(G)
C=codes.GoppaCode(g,L)
print("")
print(C.generator_matrix())
```

$$\left(\begin{array}{cccccccccccc|cccc}
1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\
1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\
0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\
0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1
\end{array}\right)$$

$$\left(\begin{array}{cccccccccccccccc}
1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\
0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1
\end{array}\right)$$

- Construct a (n, k, t) **Goppa code**

McEliece Encryption Scheme

Key Generation : Private Keys

- Construct a (n, k, t) **Goppa code**
- Let G be a generator matrix for the Goppa code.

McEliece Encryption Scheme

Key Generation : Private Keys

- Construct a (n, k, t) **Goppa code**
- Let G be a generator matrix for the Goppa code.
- Choose :

McEliece Encryption Scheme

Key Generation : Private Keys

- Construct a (n, k, t) **Goppa code**
- Let G be a generator matrix for the Goppa code.
- Choose :
  - S : A random nonsingular matrix of size $k \times k$.

McEliece Encryption Scheme

Key Generation : Private Keys

- Construct a (n, k, t) **Goppa code**
- Let G be a generator matrix for the Goppa code.
- Choose :
  - S : A random nonsingular matrix of size $k \times k$.
  - P : A random permutation matrix of size $n \times n$.

McEliece Encryption Scheme

Key Generation : Private Keys

- Construct a (n, k, t) **Goppa code**
- Let G be a generator matrix for the Goppa code.
- Choose :
  - S : A random nonsingular matrix of size $k \times k$.
  - P : A random permutation matrix of size $n \times n$.
- Private key : G, S, P, and the decoding algorithm for the Goppa code.

Masking the Generator Matrix

Key Generation : Public Keys

The McEliece encryption scheme hides the structure of the generator matrix G using two secret matrices S and P :

- S : A random nonsingular matrix of size $k \times k$.

Masking the Generator Matrix

Key Generation : Public Keys

The McEliece encryption scheme hides the structure of the generator matrix G using two secret matrices S and P :

- S : A random nonsingular matrix of size $k \times k$.
- P : A random permutation matrix of size $n \times n$.

Masking the Generator Matrix

Key Generation : Public Keys

The McEliece encryption scheme hides the structure of the generator matrix G using two secret matrices S and P :

- S : A random nonsingular matrix of size $k \times k$.
- P : A random permutation matrix of size $n \times n$.
- The public generator matrix is computed as :

$$G' = SGP$$

Masking the Generator Matrix

Key Generation : Public Keys

The McEliece encryption scheme hides the structure of the generator matrix G using two secret matrices S and P :

- S : A random nonsingular matrix of size $k \times k$.
- P : A random permutation matrix of size $n \times n$.
- The public generator matrix is computed as :

$$G' = SGP$$

- The error capacity of the code t is also public.

Masking the Generator Matrix

Key Generation : Public Keys

The McEliece encryption scheme hides the structure of the generator matrix G using two secret matrices S and P :

- S : A random nonsingular matrix of size $k \times k$.
- P : A random permutation matrix of size $n \times n$.
- The public generator matrix is computed as :

$$G' = SGP$$

- The error capacity of the code t is also public.

Purpose : This ensures that the attacker sees G' instead of G, masking the underlying structure of the code.

Encryption Process

- Let m be the plaintext message, represented as a vector of length k.

Encryption Process

- Let m be the plaintext message, represented as a vector of length k.
- Generate a random error vector e of weight t.

Encryption Process

- Let m be the plaintext message, represented as a vector of length k.
- Generate a random error vector e of weight t.
- Compute the ciphertext :

$$c = mG' + e$$

Encryption Process

- Let m be the plaintext message, represented as a vector of length k.
- Generate a random error vector e of weight t.
- Compute the ciphertext :

$$c = mG' + e$$

- Transmit c to the recipient.

Goppa Codes and Decoding

- McEliece uses **Goppa codes** as the underlying structure of the code.

Goppa Codes and Decoding

- McEliece uses **Goppa codes** as the underlying structure of the code.
- Goppa codes possess a **polynomial-time decoding algorithm**.

Goppa Codes and Decoding

- McEliece uses **Goppa codes** as the underlying structure of the code.
- Goppa codes possess a **polynomial-time decoding algorithm**.
- Complexity of the decoding algorithm :

$$O(nt^2)$$

Goppa Codes and Decoding

- McEliece uses **Goppa codes** as the underlying structure of the code.
- Goppa codes possess a **polynomial-time decoding algorithm**.
- Complexity of the decoding algorithm :

$$O(nt^2)$$

- Review :

Goppa Codes and Decoding

- McEliece uses **Goppa codes** as the underlying structure of the code.
- Goppa codes possess a **polynomial-time decoding algorithm**.
- Complexity of the decoding algorithm :

$$O(nt^2)$$

- Review :
  - n : Code length.

Goppa Codes and Decoding

- McEliece uses **Goppa codes** as the underlying structure of the code.
- Goppa codes possess a **polynomial-time decoding algorithm**.
- Complexity of the decoding algorithm :

$$O(nt^2)$$

- Review :
  - n : Code length.
  - t : Error correction capability.

Goppa Codes and Decoding

- McEliece uses **Goppa codes** as the underlying structure of the code.
- Goppa codes possess a **polynomial-time decoding algorithm**.
- Complexity of the decoding algorithm :

$$O(nt^2)$$

- Review :
  - n : Code length.
  - t : Error correction capability.

Advantage : Efficient decoding for legitimate users while maintaining security against attackers.

Decryption Process

- Receive the ciphertext c.

Decryption Process

- Receive the ciphertext c.
- Apply the inverse permutation P^{-1} to c to obtain :

$$\tilde{c} = cP^{-1} = m(SGP)P^{-1} + eP^{-1} = (mS)G + eP^{-1}, \text{ with } \omega(eP^{-1}) = t.$$

Decryption Process

- Receive the ciphertext c.
- Apply the inverse permutation P^{-1} to c to obtain :

$$\tilde{c} = cP^{-1} = m(SGP)P^{-1} + eP^{-1} = (mS)G + eP^{-1}, \text{ with } \omega(eP^{-1}) = t.$$

- Decode $\tilde{c}$ using the private decoding algorithm of the Goppa code linked to G to find mS.

Decryption Process

- Receive the ciphertext c.
- Apply the inverse permutation P^{-1} to c to obtain :

$$\tilde{c} = cP^{-1} = m(SGP)P^{-1} + eP^{-1} = (mS)G + eP^{-1}, \text{ with } \omega(eP^{-1}) = t.$$

- Decode $\tilde{c}$ using the private decoding algorithm of the Goppa code linked to G to find mS.
- Recover the original message m by applying S^{-1} :

$$m = (mS)S^{-1}$$

Indistinguishability of G'

- An attacker knows $c = mG' + e$ and G'.

Indistinguishability of G'

- An attacker knows $c = mG' + e$ and G'.
- The matrix G' is computationally indistinguishable from a random matrix of size $k \times n$.

Indistinguishability of G'

- An attacker knows $c = mG' + e$ and G'.
- The matrix G' is computationally indistinguishable from a random matrix of size $k \times n$.
- As a result, the attacker must solve the **General Decoding Problem (GDP)** :

Indistinguishability of G'

- An attacker knows $c = mG' + e$ and G'.
- The matrix G' is computationally indistinguishable from a random matrix of size $k \times n$.
- As a result, the attacker must solve the **General Decoding Problem (GDP)** :
  - Given a random generator matrix G', a received vector r, and a weight t, find the error vector e of weight t, and the vector m such that :

$$r = mG' + e$$

Indistinguishability of G'

- An attacker knows $c = mG' + e$ and G'.
- The matrix G' is computationally indistinguishable from a random matrix of size $k \times n$.
- As a result, the attacker must solve the **General Decoding Problem (GDP)** :
  - Given a random generator matrix G', a received vector r, and a weight t, find the error vector e of weight t, and the vector m such that :

$$r = mG' + e$$

  - Notice that solving GDP is equivalent to solve SD.

Indistinguishability of G'

- An attacker knows $c = mG' + e$ and G'.
- The matrix G' is computationally indistinguishable from a random matrix of size $k \times n$.
- As a result, the attacker must solve the **General Decoding Problem (GDP)** :
  - Given a random generator matrix G', a received vector r, and a weight t, find the error vector e of weight t, and the vector m such that :

$$r = mG' + e$$

  - Notice that solving GDP is equivalent to solve SD.
- **Fact** : The GDP is NP-complete, making it computationally infeasible to solve for large parameters.

Parameters Used in Post-Quantum Cryptography

param	mem	Prange	Stern	Dumer	BC	BJMM	pdw	MO	BM	
348864	0	173.4	151.4	151.4	151.5	141.9	143.4	140.8	142.7	
348864	1/3	176.7	159.3	159.7	159.7	159.1	157.8	156.4	157.5	≈AES-128
348864	1/2	178.3	162.8	163.2	163.2	162.2	160.3	158.6	159.8	
460896	0	216.8	193.3	193.2	193.3	180.8	182.6	179.8	182.1	
460896	1/3	220.4	201.7	202.2	202.2	201.6	200.6	198.6	199.8	≈AES-192
460896	1/2	222.1	205.3	205.8	205.8	204.8	203.4	201.0	202.2	
6688128	0	295.7	268.0	267.9	268.0	247.3	248.8	246.2	249.6	
6688128	1/3	299.4	279.2	279.5	279.5	278.9	277.6	274.9	276.4	≈AES-256
6688128	1/2	301.2	283.0	283.3	283.3	282.3	280.4	278.2	279.4	
6960119	0	296.8	268.4	268.3	268.4	246.6	248.4	245.7	249.2	
6960119	1/3	300.4	280.1	280.5	280.5	279.4	278.6	275.8	277.3	≈AES-256
6960119	1/2	302.2	283.9	284.3	284.3	283.3	281.4	279.2	280.3	
8192128	0	334.1	303.2	307.4	303.4	277.7	279.1	275.6	281.1	
8192128	1/3	337.7	316.8	317.1	317.1	316.3	315.5	311.9	313.3	≈AES-256
8192128	1/2	339.5	320.7	321.0	321.0	320.0	318.6	315.6	317.0	